Table of Contents

Introduction ---- 2
Getting started ---- 4
Smart UI Scaler editor window ---- 8
Smart UI Canvas Scaler component ---- 12
UI Scaler groups ---- 14
Smart UI Expander component ---- 17
Smart UI Size Limiter component ---- 20
Smart UI Aspect Fitter component ---- 21
Smart UI Aspect Switch component ---- 24
3D objects in the UI ---- 29
Smart UI Mesh Fitter component ---- 32
Smart UI World Follower component ---- 35
Smart UI Framing Bounds component ---- 39
Custom UI Scaler ---- 43
Staying in sync with UI Scalers ---- 45
Lifecycle and update order ---- 47
Scripting define ---- 50
Demo scenes ---- 52
Release notes ---- 55

Introduction

Smart UI Scaler makes one Unity UI (uGUI) layout fit every screen. Design your UI once at a reference resolution, and Smart UI Scaler fits it into any screen size, aspect ratio, safe area or camera viewport, growing it toward the edges exactly as far as you allow. No more choosing between a layout that is cut off on phones and one that floats in the middle of an ultra wide monitor.

Why not the Canvas Scaler?

Unity's Canvas Scaler scales the whole canvas by one rule. Everything stretches or shrinks together, wide screens get empty margins or cropped edges, the safe area is your problem, and a single canvas cannot serve a split screen. Smart UI Scaler works per Rect Transform instead: each scaler is a rectangle with its own reference resolution and its own rules, and you can have as many of them as your screen needs.

What you get

- 📐 **Design once, ship everywhere**
  Build the UI at your reference resolution and keep it readable from a phone in portrait to a 32:9 monitor.

- 🛡️ **Safe area done for you**
  The UI stays inside the notch and home indicator insets, follows rotation and can stay centered when the inset is only on one side, while bars and backgrounds still reach the real screen edges.

- 📏 **You decide how the UI expands**
  Choose how far the layout grows from its designed size toward the full screen and where it sits on the screen, and keep parts of it, such as a lobby or a card grid, between two aspect ratios.

- 🔀 **A layout per screen shape**
  When one layout cannot serve every screen, give phones in portrait, tablets and wide monitors each a layout of their own. The right one is picked by the shape of the screen, and you edit each layout in its real shape. Images and backgrounds keep their proportions on every screen, never stretched.

- 🧊 **3D objects in the UI**
  Lit 3D backgrounds, titles, spinning items and characters keep their place and size on every screen, like any other UI element. A character or an item can also be shown through a camera of its own, and it stays framed while it animates or changes equipment.

- 🎚️ **A UI scale slider for your players**
  Let players make the UI smaller or larger while it keeps covering the same area. Bind it to a settings slider and the whole HUD follows.

- 🧩 **Several scalers on one screen**
  HUD, menus, popups and split screen viewports each scale on their own terms. Scalers can be

nested and mixed with regular UI in the same canvas.

- 💾 **Shared settings that persist**
  Group scalers through assets, change their options at runtime and keep them between sessions with the built-in PlayerPrefs persistence or your own save system.

- 🛠️ **See it before you press Play**
  The layout looks the same in Edit Mode and in Play Mode. Preview any aspect ratio, simulate safe areas and switch between device presets from one editor window.

- ⚡ **Nothing to pay per frame**
  Scalers resize only when the screen, the safe area or their options change. In a steady frame the plugin only checks that nothing changed, with no allocations.

- 🔌 **Extensible**
  Write custom scalers for render textures, read scaler sizes in the same frame through an update event, and learn from seventeen demo scenes that cover every feature.

What is included

- Smart UI Canvas Scaler, Smart UI Expander, Smart UI Size Limiter, Smart UI Aspect Fitter and Smart UI Aspect Switch components
- Smart UI Mesh Fitter, Smart UI World Follower and Smart UI Framing Bounds components for 3D objects in the UI
- UI Scaler Groups with runtime API and PlayerPrefs persistence
- Smart UI Scaler editor window with safe area simulation and presets
- Seventeen demo scenes, from a single scaler to 3D objects in the UI and custom scalers
- Full source code, online manual and scripting API, offline PDFs

Requirements

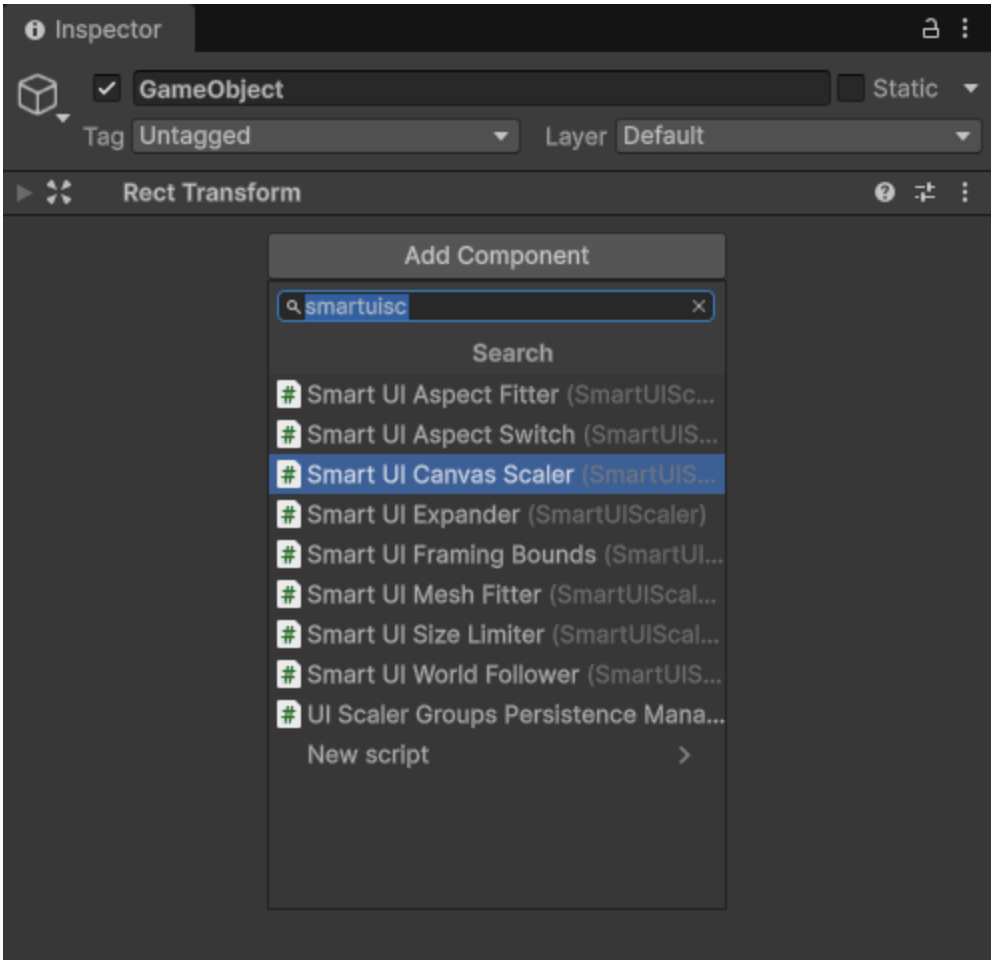
- ❗ **Unity 2022.3.0 or newer**, tested up to Unity 6000.6
- ❗ **Unity UI (uGUI)** canvases, Screen Space Overlay and Screen Space Camera. UI Toolkit is not supported.

Getting started

This guide walks you through setting up your first UI Scaler. In this manual a UI Scaler is a Rect Transform driven by a Smart UI Canvas Scaler component, or by a custom UI Scaler, and the components placed under it, such as the Smart UI Expander, are UI Scaler behaviours.

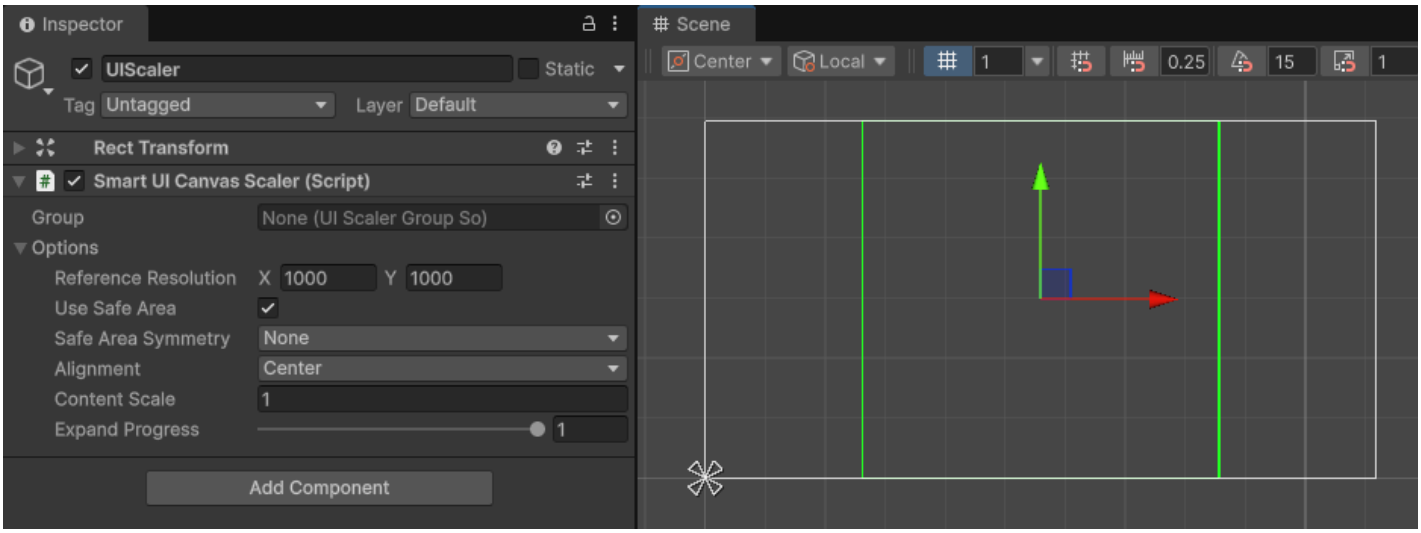
1. Create a new GameObject inside your Canvas;

2. Add the Smart UI Canvas Scaler component to this GameObject. The Rect Transform of this GameObject will be driven by the UI Scaler and will automatically resize;

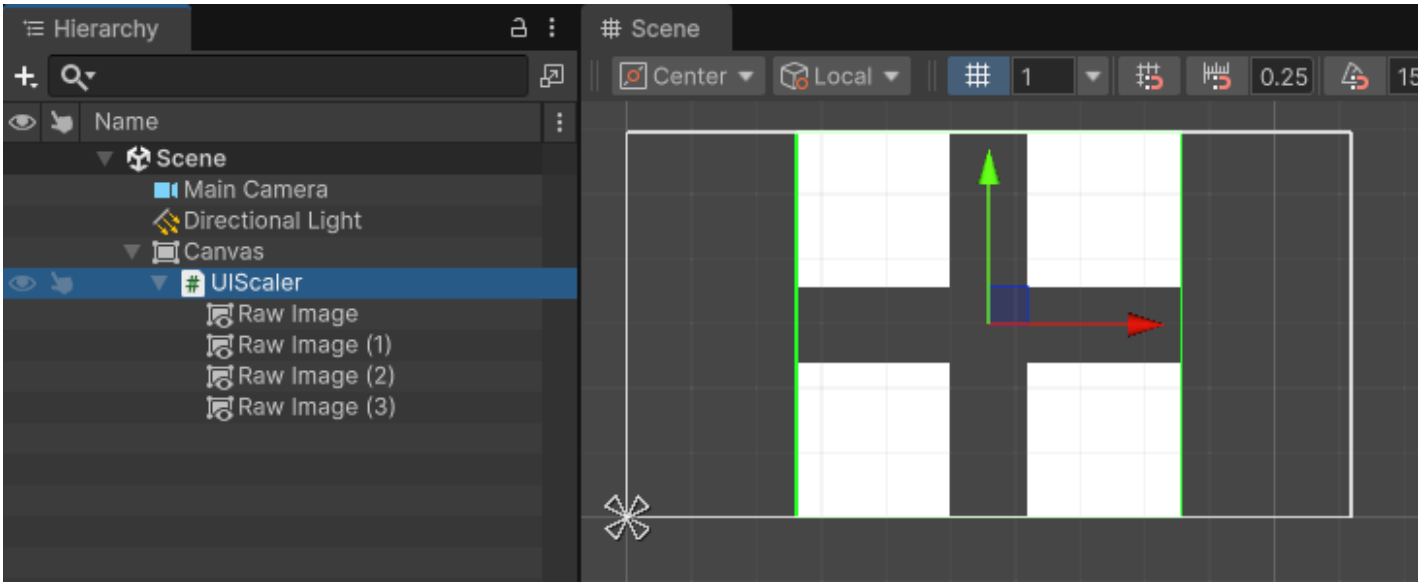


3. At first the UI Scaler will work in "Fitted Reference Resolution" mode. You can see in Scene View that the Canvas is drawn with a gray border and the UI Scaler with a green border. This mode is

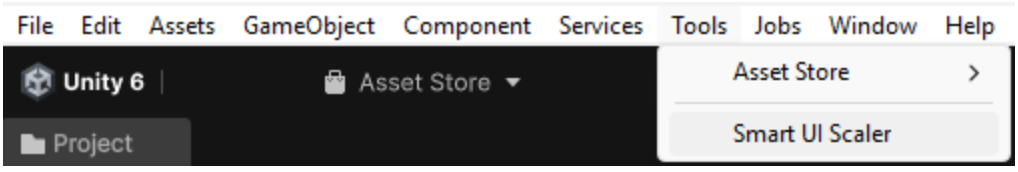
recommended for editing UI layouts inside the UI Scaler, because the layout keeps its designed size;



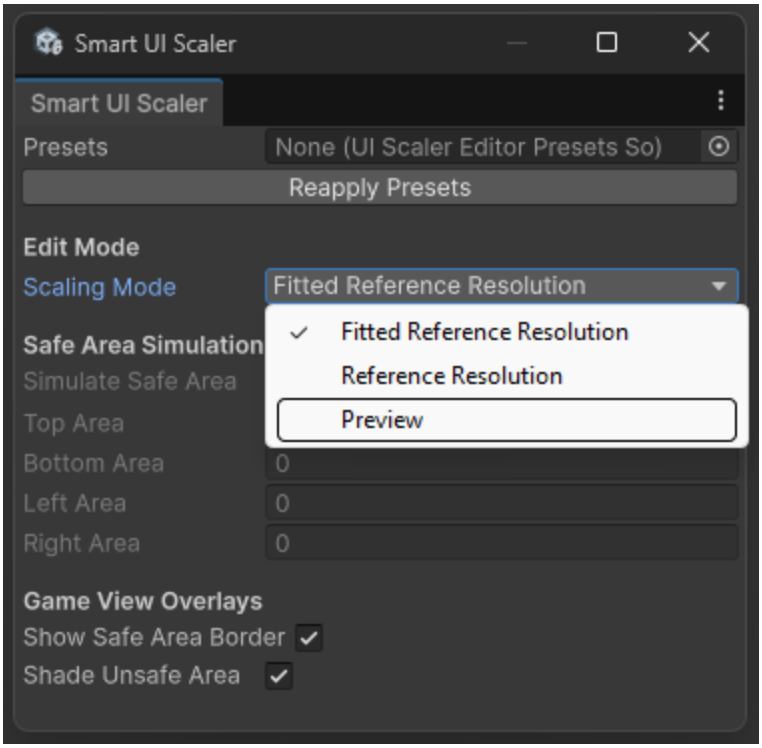
4. Let's add some UI objects to our UI Scaler. As an example, here are four Raw Images anchored to the corners of the UI Scaler;



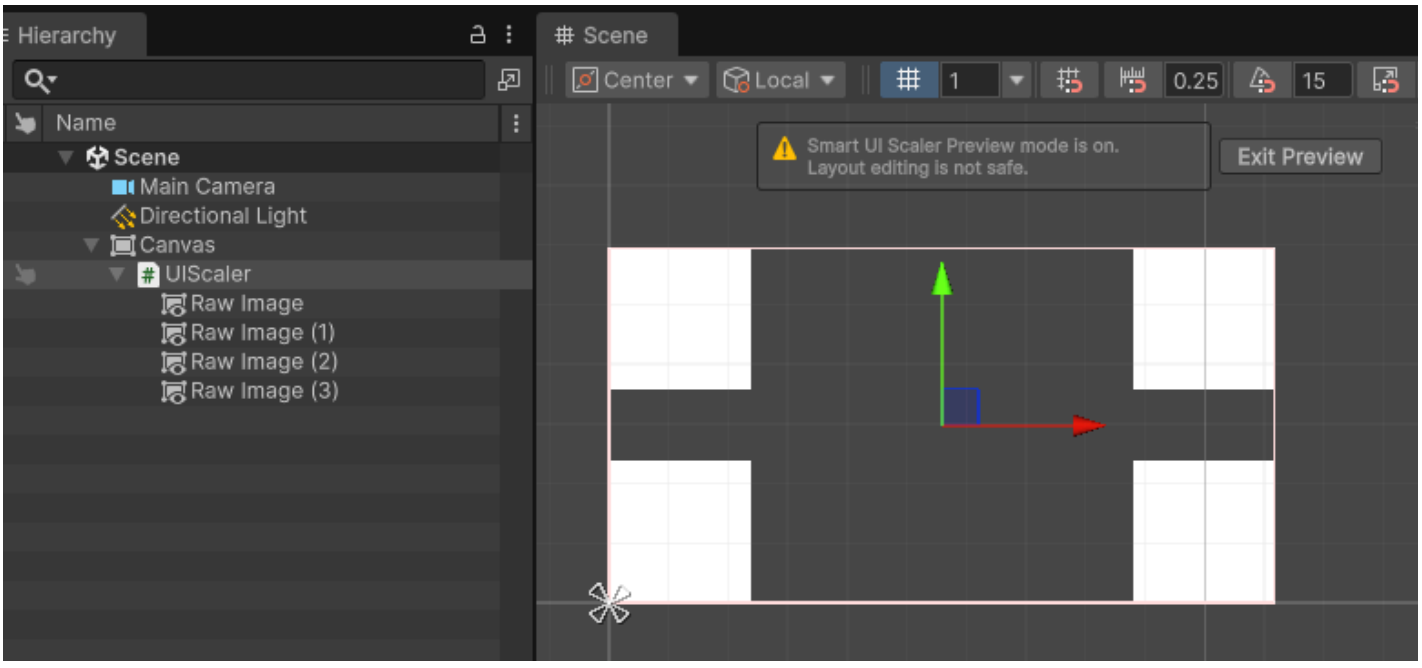
5. Now to test the UI Scaler with our UI layout open the "Smart UI Scaler" editor window under "Tools" > "Smart UI Scaler";



6. This window helps to test scaling without entering Play Mode. Change the Scaling Mode to "Preview". This mode works exactly as it would in Play Mode;



7. Our UI Scaler now takes up the full area of the Canvas. Now you can test all the Smart UI Canvas Scaler component options to see them in effect;



NOTE

In "Preview" mode the UI Scaler border turns red and the Scene view shows a reminder. This is a reminder not to edit the inner UI layout in this mode, because the layout is scaled and its size depends on the Game view.

8. Change back the "Scaling Mode" in "Smart UI Scaler" editor window to "Fitted Reference Resolution". The UI Scaler resizes back to the reference resolution and you can continue to work on your UI layout.

NOTE

The Smart UI Canvas Scaler replaces Unity's Canvas Scaler component for everything inside the UI Scaler. The Canvas Scaler can stay on the Canvas, it has no effect on the UI Scaler and its content, but it is no longer needed and can be removed.

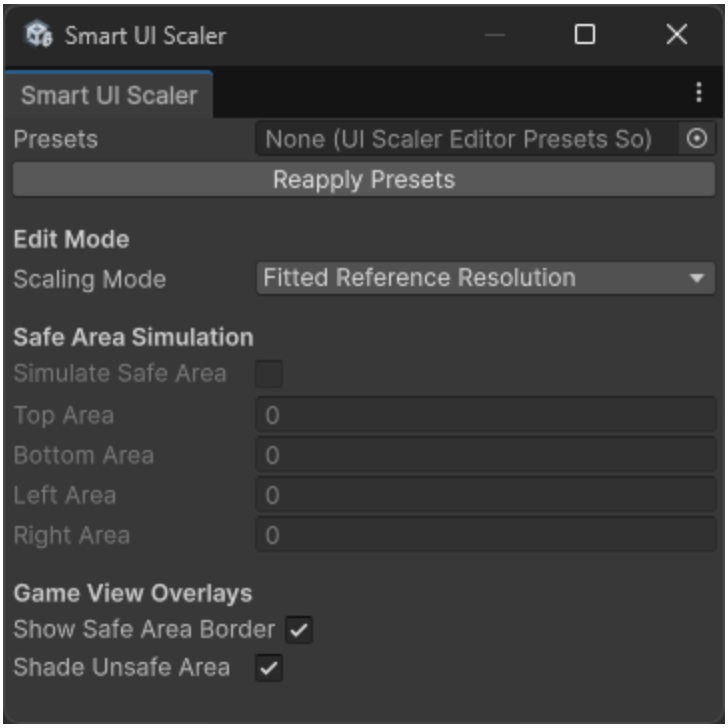
NOTE

As an alternative, closing the "Smart UI Scaler" window also resets the "Scaling Mode" to "Fitted Reference Resolution" and turns the safe area simulation and the safe area border off.

These steps cover the basics of creating and testing a UI layout. The demo scenes show every option in action.

Smart UI Scaler editor window

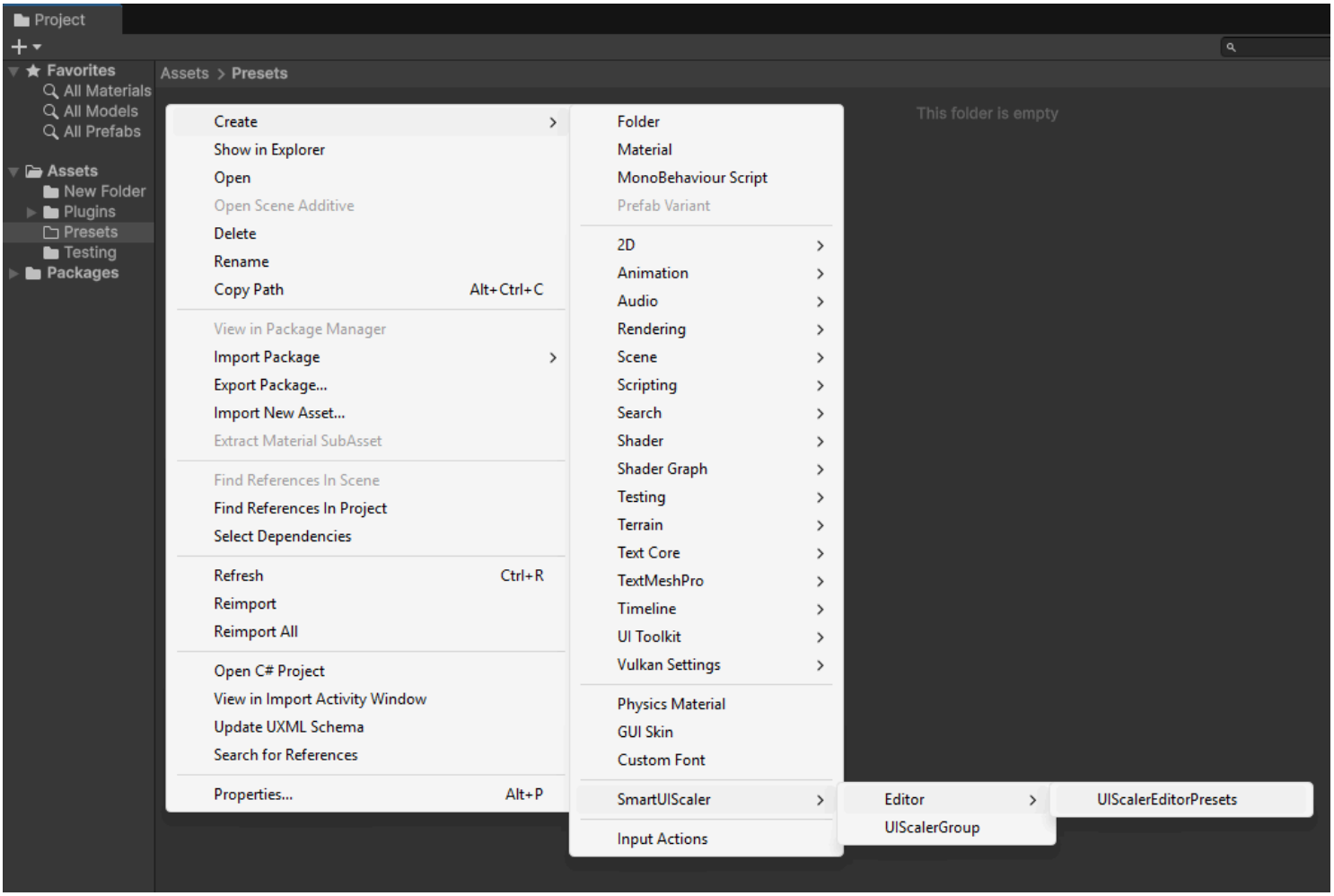
The Smart UI Scaler window helps to test and debug UI layouts without entering Play Mode. It can be found under "Tools" > "Smart UI Scaler". Its fields are grouped into the "Edit Mode", "Safe Area Simulation" and "Game View Overlays" sections, and every field has a tooltip.



Presets field

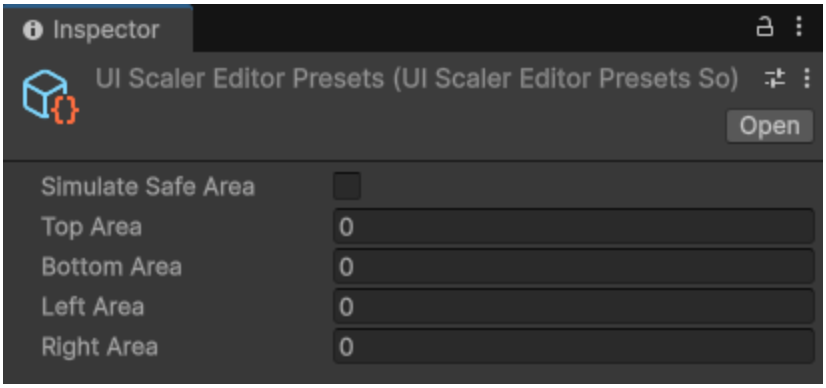
Assign a presets scriptable object to this field to fill in the window from it.

To create a new presets scriptable object Right Click on Project Window then navigate to "SmartUIScaler" > "Editor" > "UIScalerEditorPresets".



> **WARNING**
>
> UIScalerEditorPresets scriptable object should be placed inside "Editor" folder as this scriptable object only works in Editor.

A presets scriptable object describes a device: it has the Simulate Safe Area and Area options of the Smart UI Scaler window. Applying it also switches the Scaling Mode to "Preview", the only Edit Mode scaling mode that shows the simulated safe area. "Show Safe Area Border" and "Shade Unsafe Area" are set in the window only.



The presets are applied as soon as the scriptable object is selected and can be applied again with the "Reapply Presets" button, for example after the fields were changed by hand.

The plugin ships with three presets in `Demos/Common/Editor`: "No Safe Area Presets", "iPhone 12 Portrait Presets" and "iPhone 12 Landscape Presets". "No Safe Area Presets" previews the screen without a safe area, the iPhone presets enable the safe area simulation with the notch and home indicator paddings of that device.

Scaling Mode field

Scaling Mode field controls how UI Scalers are scaled in Edit Mode.

Scaling Mode	Description
Fitted Reference Resolution	Sets the size of UI Scalers to reference resolution and scales them to fit inside their Canvases. This mode is recommended for UI development and is most convenient to work with.
Reference Resolution	Sets the size of UI Scalers to reference resolution and keeps the scale to represent their reference resolution 1:1 in Unity units. This mode is also intended for UI development.
Preview	Enables full UI Scalers resizing that represents how UI Scalers will look like in Play Mode. It is not recommended to edit the inner UI layout when this mode is enabled.

> **NOTE**
>
> This field only works in Edit Mode and is hidden in Play Mode. The "Fitted Reference Resolution" and "Reference Resolution" modes center the UI Scaler inside its Canvas and ignore the Alignment option, which only applies in "Preview" mode and in Play Mode.

While Scaling Mode is set to "Preview" in Edit Mode, the Scene view shows a reminder that layout editing is not safe, with an "Exit Preview" button that switches back to "Fitted Reference Resolution" for editing.

Simulate Safe Area and Area fields

These fields control the simulated screen safe area. To activate the safe area simulation enable the "Simulate Safe Area" toggle. The Area fields control the virtual displays borders padding around the safe area. The toggle and the Area fields can only be edited in the "Preview" scaling mode and in Play Mode, and the Area fields only while "Simulate Safe Area" is enabled. An Area field cannot go past the opposite edge of the Game view: the Top Area is at most the Game view height minus the Bottom Area, and the same for the other sides.

> **NOTE**
>
> Safe area simulation fields also work in Play Mode.

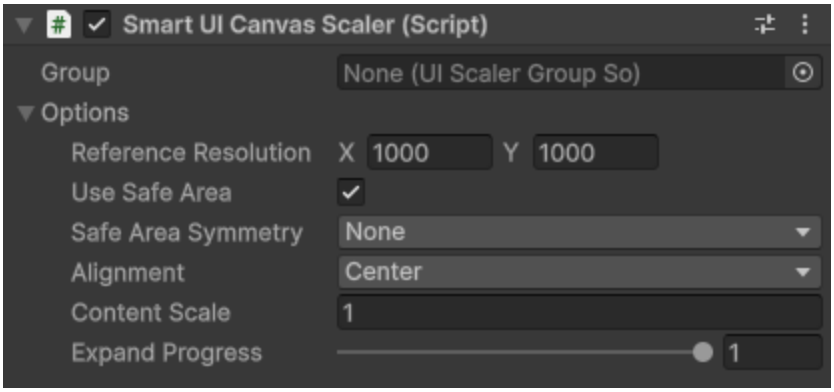
Show Safe Area Border field

Draws the safe area in the Game view: a border around it and a shade over the margins outside it, on top of all Canvases. The shade can be turned off with the "Shade Unsafe Area" toggle below it. With "Simulate Safe Area" enabled, in the "Preview" scaling mode and in Play Mode, it shows the simulated safe area. Otherwise it shows the whole Game view, or the safe area of the simulated device in the Device Simulator. It is only drawn in the Editor, in Edit Mode and in Play Mode, and never in builds.

> **NOTE**
>
> The border and the shade are drawn on top of the Game view, so they also appear in screenshots taken with ScreenCapture. Turn "Show Safe Area Border" off before taking screenshots.

Smart UI Canvas Scaler component

The main component of this plugin that replaces the built-in Canvas Scaler component. It is placed on a child of a Canvas.



Group field

Used to assign this UI Scaler into a shared group with shared options. For more information see UI Scaler groups.

Options fields

These options are used when this UI Scaler is not assigned into a group.

Field	Description
Reference Resolution	The resolution your UI layout is designed for. In Edit Mode the UI Scaler has exactly this size. Minimum is 100 on each axis.
Use Safe Area	Fit into the screen's safe area instead of the whole screen area.
Safe Area Symmetry	Shown when Use Safe Area is on. Mirrors the safe area inset of one side to the opposite side, so the UI Scaler stays centered: None uses the safe area as reported, Horizontal applies the larger of the left and right insets to both sides, Vertical the larger of the top and bottom insets, Both mirrors both axes. Split screen UI Scalers with Horizontal keep the same scale when only one side has an inset.
Alignment	Where the UI Scaler sits inside the Canvas area when it does not fill it, that is when Expand Progress is below 1.
Content Scale	Additional scale applied to the children. 1 is the designed size, lower values shrink the UI while it keeps covering the same area. Minimum is 0.1, recommended range is 0.1 to 1.

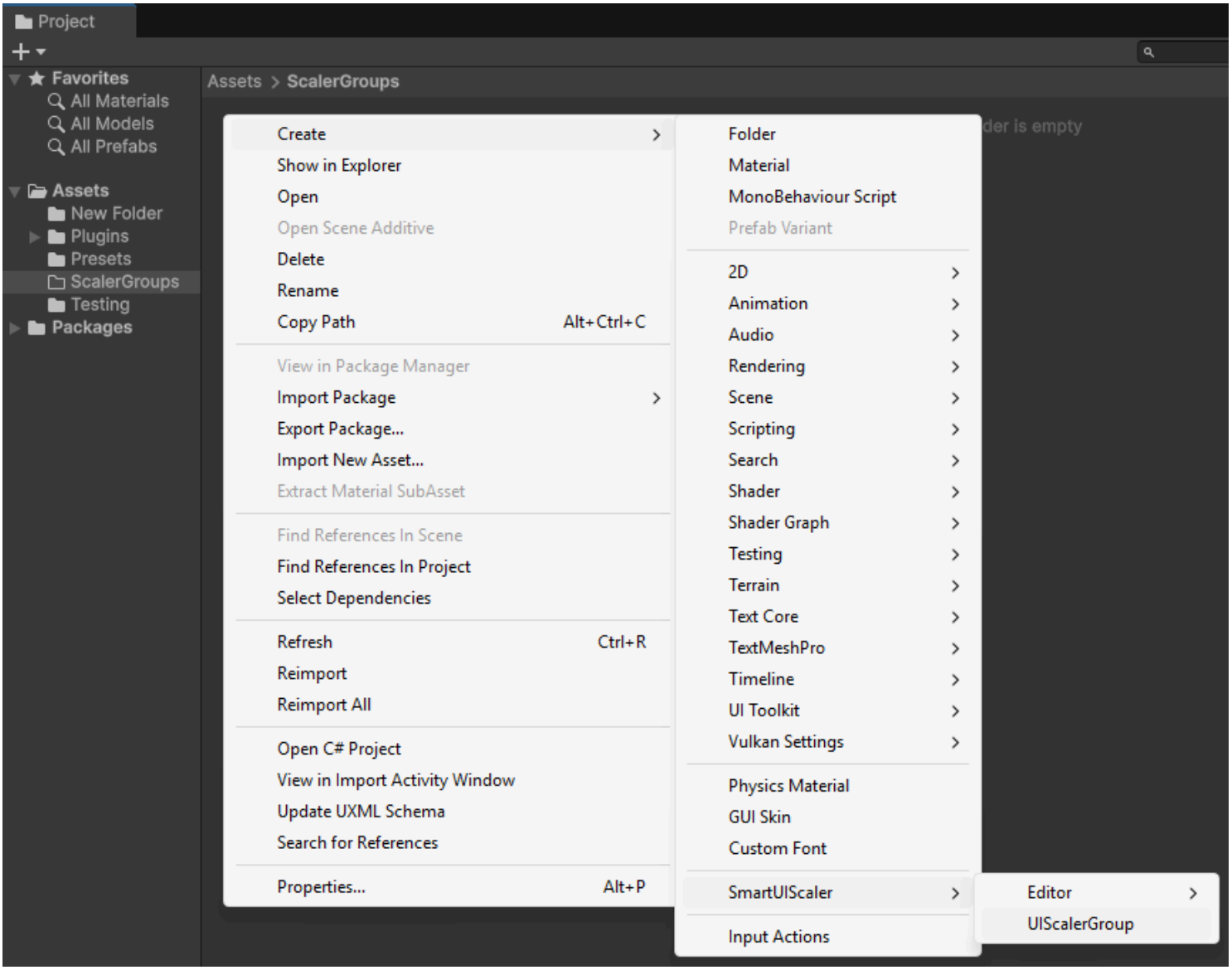
Field	Description
Expand Progress	How far the UI Scaler grows from the reference resolution (0) toward the whole available area (1). Allowed range is 0 to 1.

WARNING

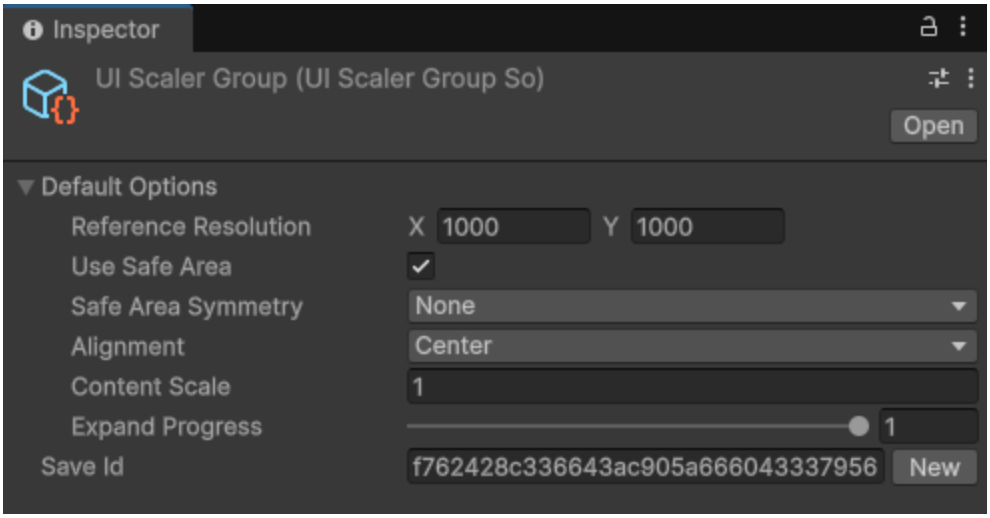
This UI Scaler only works in Canvases with "Screen Space Overlay" Render Mode or "Screen Space Camera" Render Mode where attached camera is rendering directly to the screen. When the attached camera renders to a Render Texture this UI Scaler cannot know the screen space area of the rendered image and this information has to be additionally provided.

UI Scaler groups

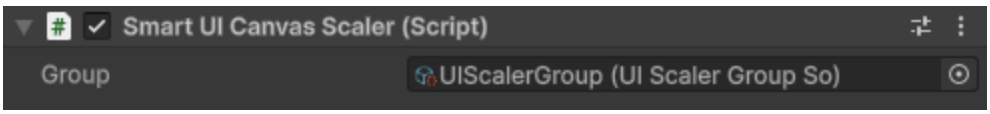
UI Scaler Groups are used to share and control UI Scaler options between UI Scalers. UI Scaler Groups are controlled through their scriptable objects. To create a UI Scaler Group scriptable object Right Click on Project Window then navigate to "SmartUIScaler" > "UIScalerGroup".



The UI Scaler Group scriptable object has a "Default Options" foldout with the same fields as the Smart UI Canvas Scaler component. These are the options every UI Scaler in the group starts with. At runtime Content Scale and Expand Progress can be changed through the scripting API, and the changed values apply to all UI Scalers in the group. Reference Resolution, Use Safe Area, Safe Area Symmetry and Alignment are fixed for the group.



To assign a Smart UI Canvas Scaler to this group drag the UI Scaler Group scriptable object into its "Group" field. The "Options" field disappears, as the group options are used instead.



Use one group for UI Scalers that should look like one UI, even when they sit on different canvases: the canvases of a camera stack, a HUD split over several canvases, or popups on a canvas of their own. A change of Content Scale or Expand Progress then applies to all of them at once, and they are saved as one group.

Content Scale and Expand Progress are changed with the `SetContentScale` and `SetExpandProgress` methods of the UI Scaler Group scriptable object (UIScalerGroupSo API). Once the options are changed, all active UI Scalers update automatically. Check out the Demo 6 inside the plugin, where a complete UI Scaler Group example is showcased.

Save Id field

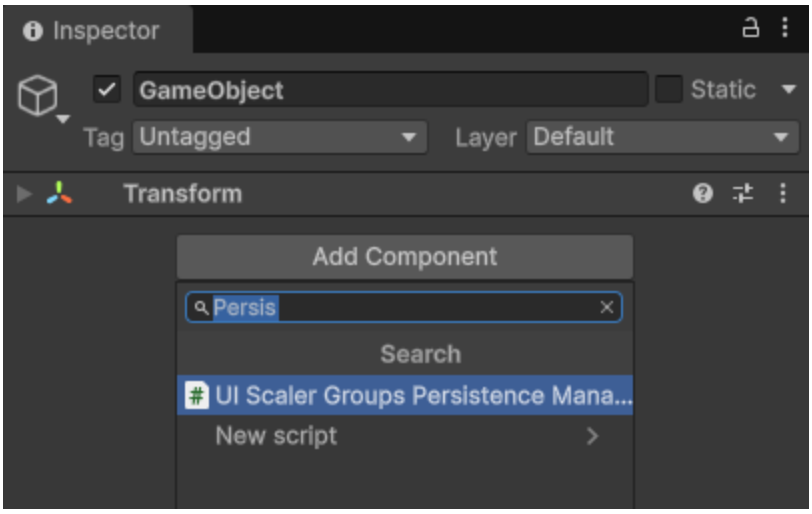
The "Save Id" field identifies the group in saved data. It is generated when the scriptable object is created, so renaming or moving the asset does not affect saved settings. The "New" button generates a fresh Save Id, which detaches the group from any settings saved under the old one.

The editor keeps Save Ids unique across the project and shows a warning when two groups share one, because only one of them would be saved.

Persistence

Initially UI Scaler Group options are not saved and loaded. To have persistent UI Scaler Group options the easiest way is to use UI Scaler Groups Persistence Manager component. Just add this component to an empty GameObject in the first scene in your project. It keeps its GameObject alive across scene loads, and a second Persistence Manager loaded later removes itself. It loads the options from PlayerPrefs

when it awakes and saves them when the game is paused or closed. The "Save Key" field is the PlayerPrefs key used for this.

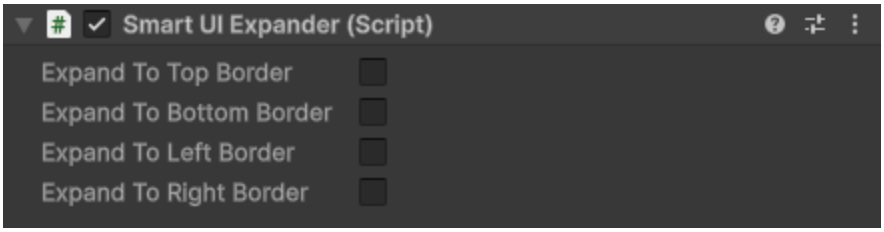


For more advanced scenarios a custom saving and loading can be implemented through UIScalerGroups class GetSaveData() and SetSaveData(UIScalerGroupsSaveData) static methods.

> **WARNING**
>
> Settings saved by version 1.0 are not restored after upgrading to 1.1.0 or later. Version 1.0 identified groups differently, so each group starts from its "Default Options" once and is saved in the new format from then on.

Smart UI Expander component

Used to expand a Rect Transform outside of the parent UI Scaler to the Canvas borders.



Field	Description
Expand To Top Border, Bottom, Left, Right	The edges that are moved to the Canvas borders.

How it works

The Smart UI Expander follows the parent Rect Transform and uses its rectangle as the base from which to expand to the Canvas borders. The four expansion fields choose the edges that reach the borders; the other edges stay on the edges of the parent. The whole Rect Transform is set by the component and recalculated on every resize, so it is greyed out in the Inspector, and turning an edge off simply puts it back on the edge of the parent.

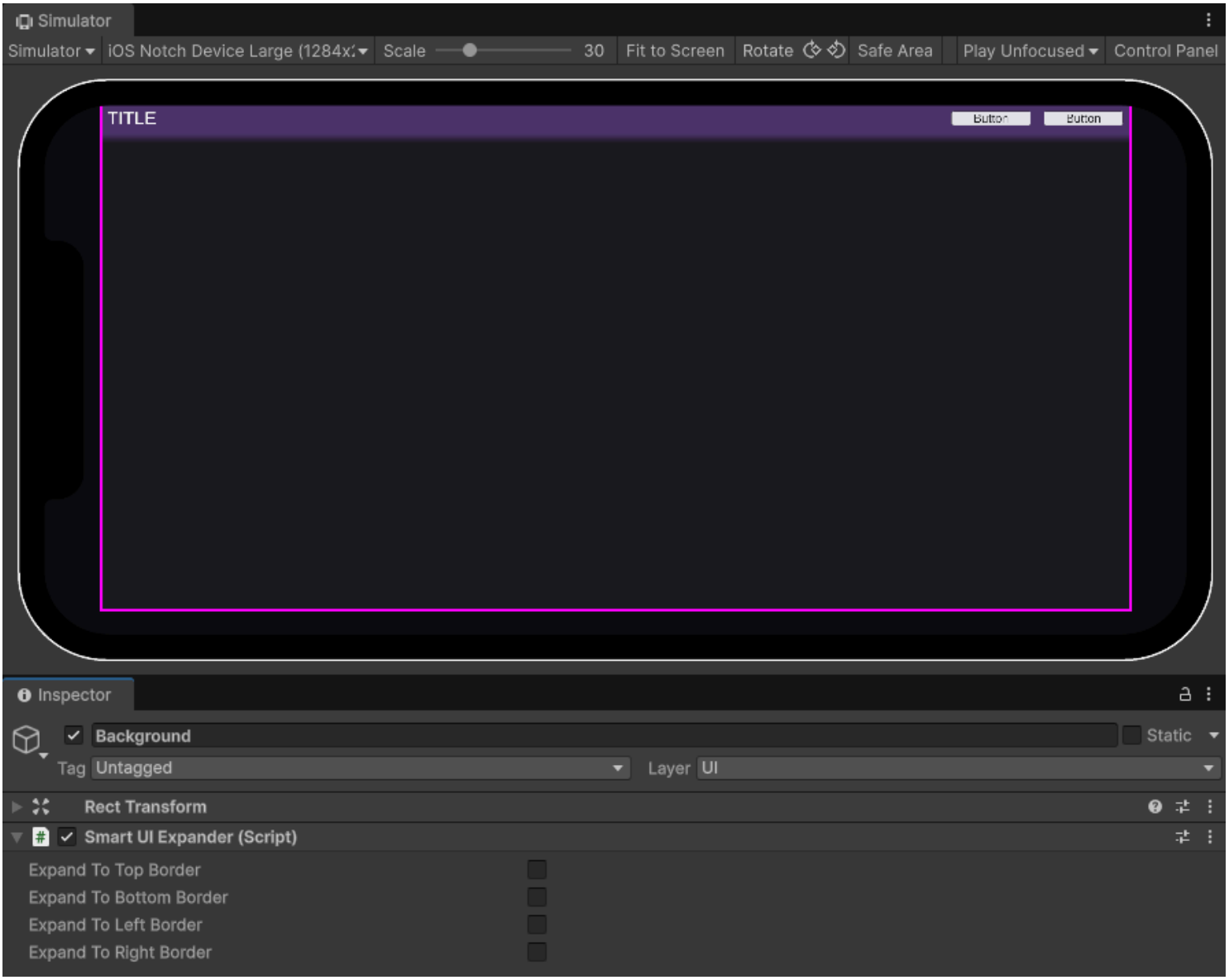
Place the Smart UI Expander on a child of the element it extends, for example the background of a header: the header Rect Transform defines the size inside the safe area, and the background child with the Smart UI Expander covers the header and reaches past it to the screen borders. Keep the header content, such as titles and buttons, next to the background, not under it, so it stays inside the safe area.

> **TIP**
>
> On screens with a large safe area margin, an expanded background gets taller or wider than designed. Use an Image with the Sliced or Tiled image type, or keep gradients and details inside the sprite's border, so the extra space is filled with the plain middle of the sprite instead of a stretched gradient.

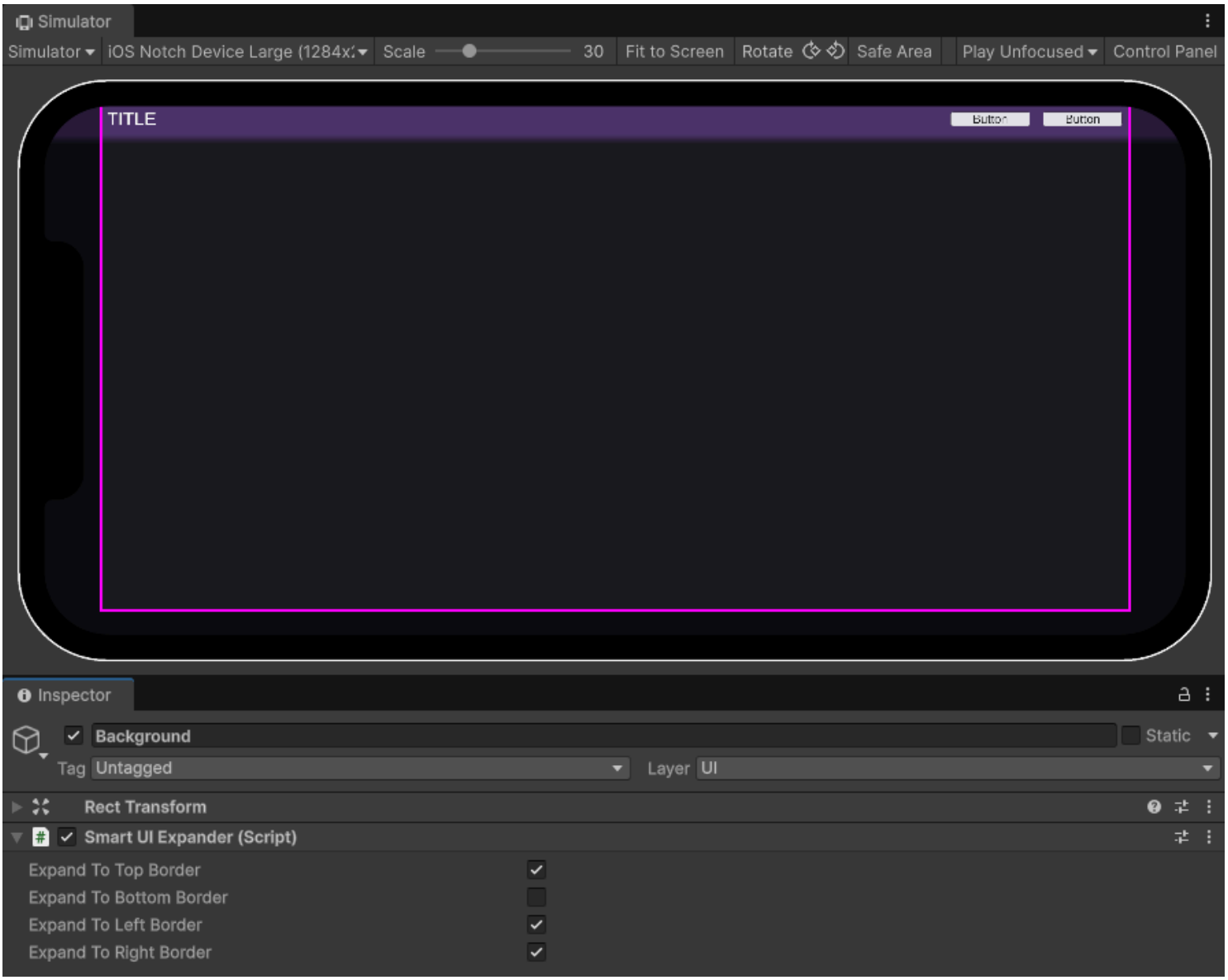
Showcase

Here is a simple example in the Device Simulator on a phone with a notch. The magenta outline is the safe area border of the Smart UI Scaler window, and the UI Scaler fits inside it. The header is at the top and its purple background has the Smart UI Expander component.

In the first image you can see that the header is inside the safe area and the Smart UI Expander does not do anything as no expansion field is selected.



After the expansion to the top, left and right borders is turned on, the header background reaches past the safe area to the screen edges, while the title and buttons stay inside the safe area.



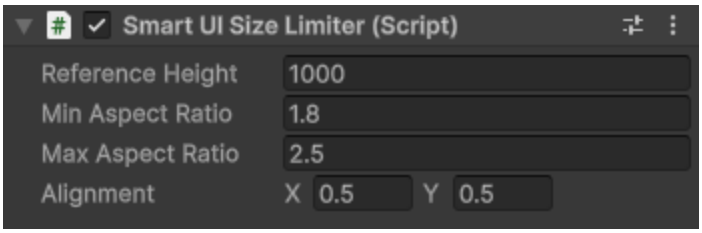
You can find the implementation of this example inside the Demo 7 of the plugin.

Placing elements against the screen edge

UI Scalers keep their content inside the safe area. To place an element against the real screen edge and still scale it with the UI, for example a corner decoration, put it under a Rect Transform with a Smart UI Expander that expands to all four borders, directly under the UI Scaler.

Smart UI Size Limiter component

Keeps a layout, such as a lobby or a card grid, between a minimum and a maximum aspect ratio inside its parent, so it neither stretches on ultrawide screens nor gets squeezed on tall ones. The Rect Transform gets a fixed reference height and an aspect ratio limited to the assigned min and max values.



Field	Description
Reference Height	Height set on the Rect Transform and used to calculate the width for the current aspect ratio. Minimum is 1.
Min Aspect Ratio	Minimum limit for aspect ratio. For unlimited min aspect ratio - set the value to 0.
Max Aspect Ratio	Maximum limit for aspect ratio. For unlimited max aspect ratio - set the value to 0.
Alignment	Where the limited Rect Transform sits inside the parent, from (0, 0) bottom left to (1, 1) top right. (0.5, 0.5) centers it, (1, 0) keeps it on the bottom right corner, values in between split the free space.

How it works

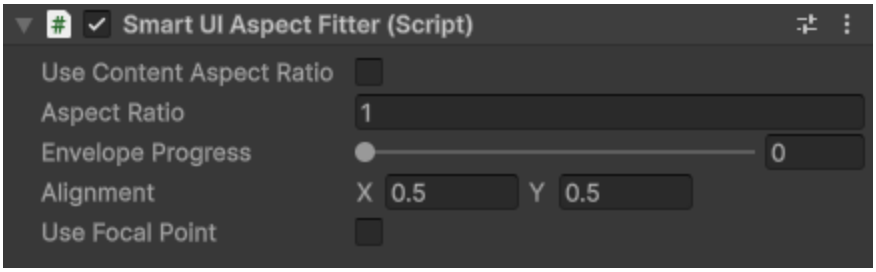
Smart UI Size Limiter component depends on parent Rect Transform. It uses the parent Rect Transform's size to calculate the aspect ratio of its own Rect Transform. If the parent's aspect ratio is between the min and max limits the full parent size is occupied, otherwise the size is limited to the nearest limit. The free space left in the parent is split by the Alignment field.

> **NOTE**
>
> Smart UI Size Limiter component can work without UI Scaler in parent hierarchy. To keep it inside the safe area, place it under a UI Scaler with Use Safe Area enabled.

Demo 8 shows a lobby layout kept between a min and a max aspect ratio.

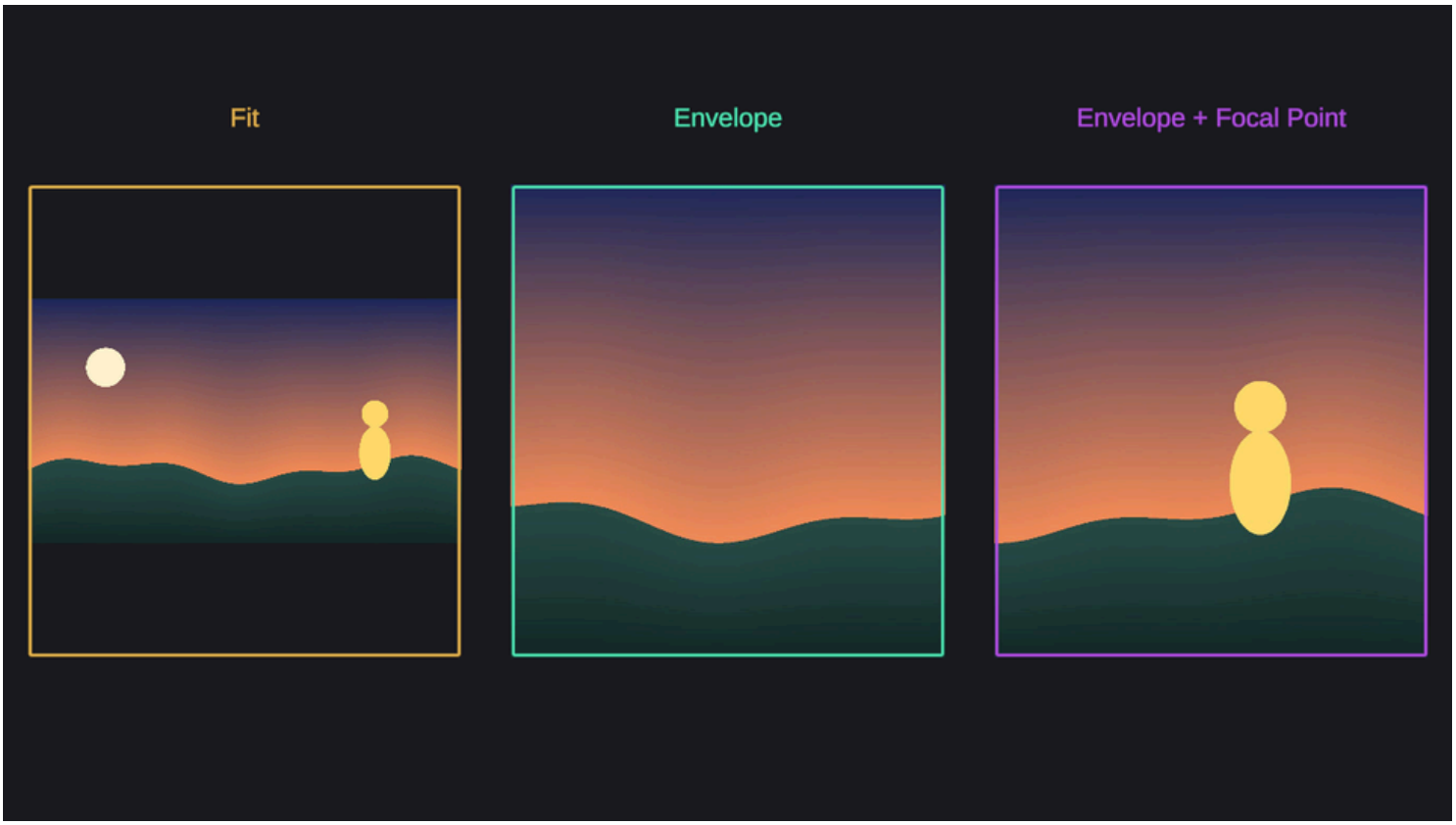
Smart UI Aspect Fitter component

Fits the Rect Transform into the parent Rect Transform with a set aspect ratio, anywhere between fitting inside the parent and covering it. Use it for images and backgrounds that must keep their proportions on every screen.



Field	Description
Use Content Aspect Ratio	Take the aspect ratio from the texture of the RawImage, cropped by its UV Rect, or the sprite of the Image on the same GameObject. It follows the content when the texture or sprite changes.
Aspect Ratio	Width divided by height. Used when Use Content Aspect Ratio is off, or when the content has no texture or sprite. Shown in the Inspector only while Use Content Aspect Ratio is off.
Envelope Progress	0 fits the Rect Transform inside the parent, the whole content is visible and empty space is left on two sides. 1 covers the whole parent, the content is cropped on two sides. Values in between do a bit of both.
Alignment	Where the Rect Transform sits inside the parent, from (0, 0) bottom left to (1, 1) top right. (0.5, 0.5) centers it, (1, 0) keeps it on the bottom right corner. When the content is larger than the parent, it decides which part is cropped.
Use Focal Point	Keep a point of the content on the Alignment point of the parent while the content covers the parent.
Focal Point	The point of the content to keep, from (0, 0) bottom left to (1, 1) top right of the content.

The same picture in three frames that follow the screen shape: Envelope Progress 0 fits the whole picture inside its frame, 1 covers the frame, and 1 with a focal point covers the frame while keeping the character in view.



Focal point

With a focal point the content is moved so that the chosen point, for example a character's face, stays on the Alignment point of the parent, which is the center by default. The content only stops following the focal point where moving further would uncover an edge of the parent.

With Use Focal Point on, a selected Aspect Fitter shows the focal point as a yellow cross in the Scene view. Drag it onto the part of the content to keep. A cyan circle marks the Alignment point it is kept on.

Without a focal point, Alignment decides alone which part of the content stays visible: (0.5, 0.5) crops evenly on both sides, (0, 0.5) keeps the left edge.

> **NOTE**
> Smart UI Aspect Fitter component can work without UI Scaler in parent hierarchy. To keep it inside the safe area, place it under a UI Scaler with Use Safe Area enabled.

Covering the whole screen

To cover the whole screen with a picture, including the area outside the safe area, use two objects under the UI Scaler:

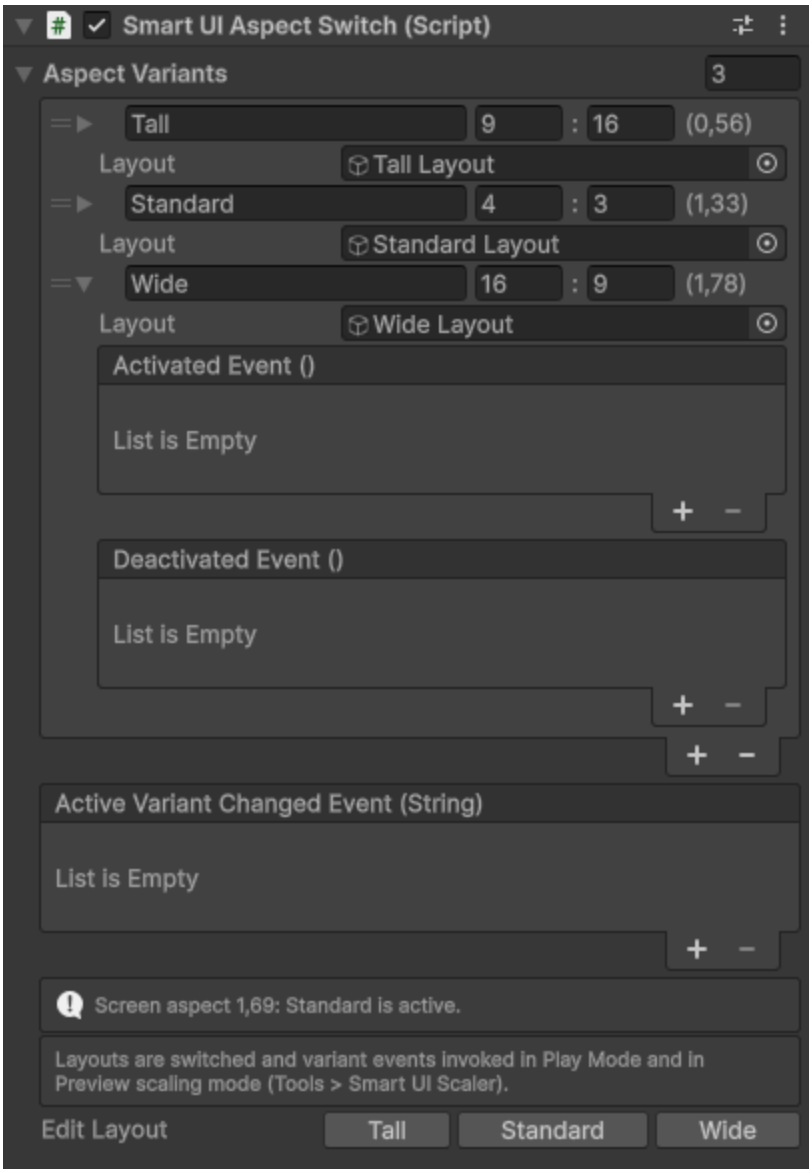
1. A Rect Transform with a Smart UI Expander that expands to all four Canvas borders. Its rect is always the whole Canvas.
2. Under it, a RawImage or Image with a Smart UI Aspect Fitter: Use Content Aspect Ratio on and Envelope Progress 1.

The picture then covers the screen on every screen shape without being stretched, and Alignment or the Focal Point decide which part stays visible.

Smart UI Aspect Switch component

Switches between layouts by the aspect ratio of the screen area of a UI Scaler. Use it when one layout cannot serve every screen shape, for example a wide layout with side panels for desktop and ultrawide screens, a standard layout for tablets and a tall layout for phones in portrait.

Add the component to a UI Scaler, or to any object under one, and add an aspect variant for each layout.



Field	Description
Name	First field of the variant row. Name of the variant, reported as the active variant name in code.
Aspect ratio	The two numbers next to the name. Smallest aspect ratio of the screen area this variant is used for, as width : height, for example 16 : 9. The value in brackets is the same ratio as a single number.

Field	Description
Layout	Object that is activated while this variant is active and deactivated otherwise. Optional.
Activated Event	Event invoked when this variant becomes active. Shown when the variant is expanded.
Deactivated Event	Event invoked when this variant stops being active. Shown when the variant is expanded.
Active Variant Changed Event	Below the variants. Invoked with the name of the new active variant whenever the active variant changes. It can call a method that takes a text directly, for example `Animator.SetTrigger` with a trigger per variant name, or the text of a label.

How the variant is chosen

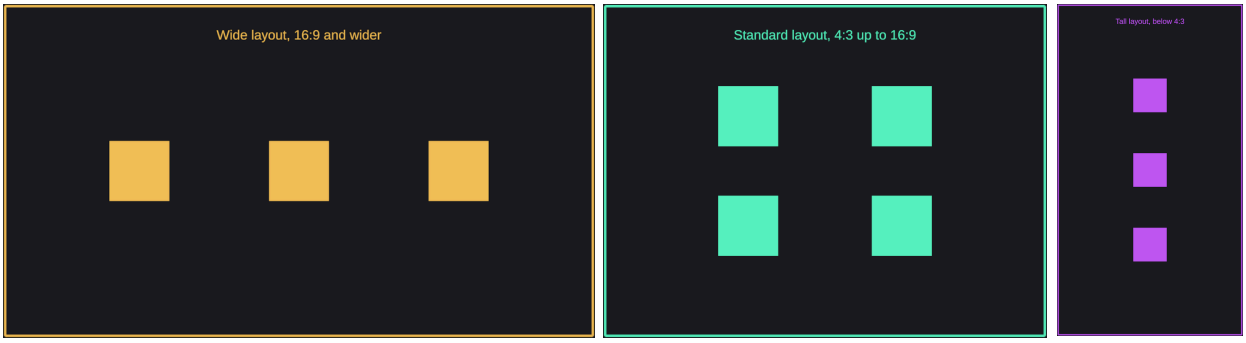
The aspect ratio of the UI Scaler's screen area is width divided by height. The screen area is the safe area when the UI Scaler uses it, so a notch or a home indicator counts. The variant with the largest Aspect Ratio that is not wider than the screen area becomes active. When the screen area is thinner than every variant, the thinnest variant is used.

Example with three variants, Tall 9 : 16, Standard 4 : 3 and Wide 16 : 9:

Screen area	Active variant
21 : 9 (2.33)	Wide
16 : 9 (1.78)	Wide
16 : 10 (1.60)	Standard
1 : 1 (1.00)	Tall
9 : 19.5 (0.46)	Tall, the thinnest variant

Keep the variants ordered from the thinnest to the widest. The inspector offers a "Sort Aspect Variants" button when they are not, and warns when two variants share the same ratio.

The same UI Scaler with these three variants on a 16:9, a 4:3 and a 9:16 screen:



Inspector

- The inspector shows the aspect ratio of the screen area and the active variant.
- Layouts are switched and the variant events invoked in Play Mode and in the "Preview" scaling mode of the Smart UI Scaler editor window. In the other scaling modes the layouts are left as they are, so each one can be edited.
- The inspector warns when no UI Scaler is found on the object or its parents, when a layout is the object itself or one of its parents, and when two variants use the same layout.

Editing each layout in its own shape

In the "Fitted Reference Resolution" and "Reference Resolution" scaling modes, the inspector has an "Edit Layout" button for each variant. It shows the layout of the variant, hides the others, and sizes the UI Scaler exactly as it is sized in Play Mode on a screen with the aspect ratio of the variant, letterboxed in the Game view. A tall layout is then edited in a tall area, with the same Expand Progress, Content Scale and Alignment as on a device. The "Reference Resolution" button returns the UI Scaler to its reference resolution.

Showing a layout can be undone, and the shape of the UI Scaler is only an editor preview: it is never saved and is cleared when the Aspect Switch is disabled, in "Preview" and in Play Mode.

> **TIP**
>
> Save the scene with all layouts inactive. A layout that is saved active runs `Awake()`, `OnEnable()`, `Start()` and one `Update()` of its scripts before the Aspect Switch deactivates it on a screen where another variant is used. With all layouts saved inactive, the Aspect Switch only activates the layout that is used, so only its scripts run. "Edit Layout" activates the layout it shows, so deactivate that layout again before saving. See Lifecycle and update order.

Reacting from code

Read the active variant with `ActiveVariant`, `ActiveVariantName` or `ActiveVariantIndex`, and get notified when it changes. The listener receives the new variant, and `PreviousVariant` tells which one was active before, so a transition can depend on its direction, for example from wide to tall:

```
using SmartUIScaler;
using UnityEngine;

public class MenuLayoutListener : MonoBehaviour
{
        [SerializeField]
        private SmartUIAspectSwitch _aspectSwitch;

        private void OnEnable()
        {
                _aspectSwitch.AddActiveVariantChangedListener(OnActiveVariantChanged);
        }

        private void OnDisable()
        {
                _aspectSwitch.RemoveActiveVariantChangedListener(OnActiveVariantChanged);
        }

        private void OnActiveVariantChanged(UIScalerAspectVariant active)
        {
                Debug.Log($"Layout changed from {_aspectSwitch.PreviousVariant.Name ??
"none"} to {active.Name}");
        }
}
```

The listener is invoked right away when the active variant is already known, so a listener added late does not miss the current state. When no variant was active before, `PreviousVariant` is empty: its `Name` is null. Pass `false` as the second argument to only hear about later changes. The variants are read-only from code: their name, aspect ratio and layout can be read with `GetVariant`, and their events are wired in the inspector. To react to one variant from code, check the name in the listener, for example `if (active.Name == "Tall")`.

When the active variant changes, the order is: the layouts are switched, the deactivated event of the previous variant is invoked, then the activated event of the new one, then the Active Variant Changed event, then the listeners added from code. All of this happens before the Canvas layout is rebuilt in the same frame. The layouts are switched before the first frame is rendered. For what the scripts in a layout that was saved active receive in the first frame, see Lifecycle and update order.

NOTE

In the editor the active variant is also resolved in the scaling modes other than "Preview", so listeners added from code are invoked there too. Only the layouts and the variant events wait for Play Mode or "Preview".

3D objects in the UI

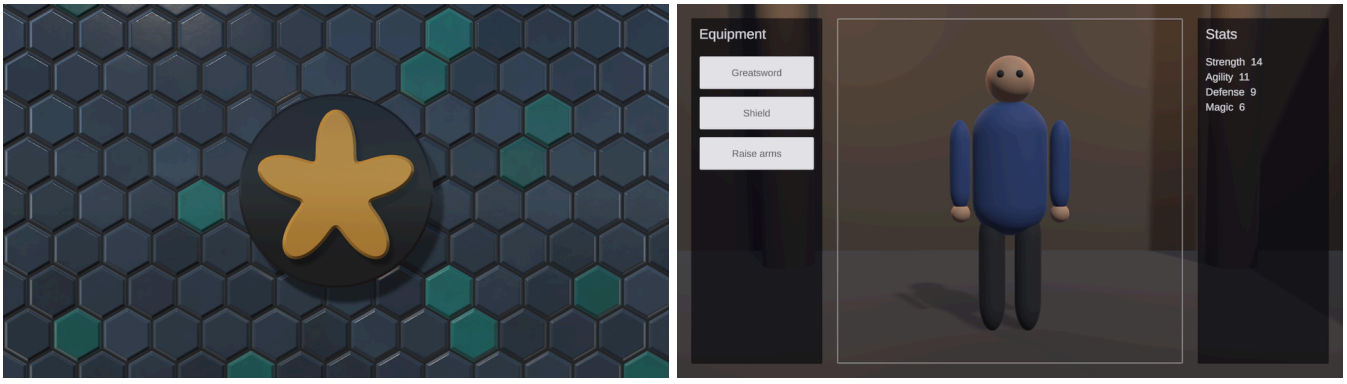
Smart UI Scaler has two components that bring 3D objects into a layout that follows every screen shape, and one component that describes the volume they fit. They look similar, but each one changes something different. This page helps you choose.

Which one do I need?

The Smart UI Mesh Fitter makes the 3D object part of the UI: it is placed and sized inside a Rect Transform like any other UI element and drawn by the camera of the canvas. The Smart UI World Follower turns a Rect Transform into a window into a 3D world: the object stays where it is in its scene, and its own camera shows it in the Rect Transform.

Many objects, such as an item in an inventory, work with both. These questions decide:

Question	**If yes**
Is the canvas Screen Space Overlay?	Smart UI World Follower. The Mesh Fitter cannot draw 3D objects on Screen Space Overlay canvases.
Should the object be lit by lights of its own or use camera effects of its own, separate from the UI? Or does it belong to a scene, such as a character standing in a room?	Smart UI World Follower
Otherwise: the canvas is Screen Space Camera or World Space, and the object is lit together with the UI	Smart UI Mesh Fitter. It needs no extra camera, so it also suits many objects at once.



Left, part of the UI: two Smart UI Mesh Fitters size the 3D objects like UI elements. The tiled wall is set to Envelope and covers the whole screen at the proportions of its texture, resizing with every screen shape, and the emblem is set to Fit in a slot that stays in the center of the UI Scaler. The canvas is Screen Space Camera, so both are lit together with the rest of the UI and the emblem casts its shadow on the wall.

Right, a window into a 3D world: the character belongs to a room with lights of its own, so a Smart UI World Follower shows it through its own camera in the free space between the menu panels.

Then, if the object changes its size through animations, poses or equipment, add Smart UI Framing Bounds to describe the volume that must always fit. It is not a third choice: it adds to either component.

Comparison

	Smart UI Mesh Fitter	**Smart UI World Follower**
What it changes	The object: its position and scale	The camera's projection, and with Move Camera also its position
Where the object lives	Under a Rect Transform inside the canvas	Anywhere in the scene
Camera	The camera of the canvas	A camera of its own, or the scene camera
Size of the object in its scene	Changes with the UI	Stays the same, the view zooms instead
Lights	Lit together with the rest of the UI	Lit by its own scene or lights of its own
Screen Space Overlay canvas	Not drawn	Works
Extra rendering cost	None	One camera per follower, unless the scene camera is used
With Smart UI Framing Bounds	The typed-in volume is fitted instead of the measured Renderer bounds, so an animated model keeps its size	The view zooms so the typed-in volume fits the Target, including equipment with framing bounds of its own
Typical use	Lit backgrounds, emblems, 3D icons, coins, chests, items in slots and item inspectors	Character screens, a character in a room between menu panels, item inspectors with lights of their own

Examples

You want	Use
A lit background that covers the screen at the proportions of its texture	Mesh Fitter set to Envelope, on a quad with the aspect ratio of its texture
A 3D logo on that background that casts a shadow on it	Mesh Fitter set to Fit, with a Depth Offset
Rotating 3D icons, such as a coin next to the coin counter or an item in each slot of a shop grid	Mesh Fitter with Use Bounding Sphere in each slot, rotating a child object. On a Screen Space Overlay canvas: World Follower with a camera of its own per slot
An item inspector where the player rotates and zooms an item, as in an RPG inventory	Mesh Fitter with Use Bounding Sphere, rotating a child object, zoomed by resizing its slot. On a Screen Space Overlay canvas or with lights of its own: World Follower with Framing Bounds set to Sphere, zoomed by changing the Radius
A 3D title, tilted or gently swaying	Mesh Fitter set to Fit in the title slot, rotating a child object. For a swaying title, Framing Bounds set to Box around its movement; Use Bounding Sphere would shrink a wide title
A character in a room that stays between the menu panels and fully visible while it equips a long sword	World Follower on the scene camera with Clip To Target and Move Camera off, plus Framing Bounds on the character and on the sword

Combining them

- **Mesh Fitter and World Follower** are not used on the same object. A scene can use both: a lit background with a Mesh Fitter behind the menu, and a character in a 3D room through a World Follower.
- **Smart UI Aspect Fitter** is the 2D counterpart: it keeps images and Rect Transforms at their aspect ratio. The Fit and Envelope modes of the Mesh Fitter do the same for 3D objects.

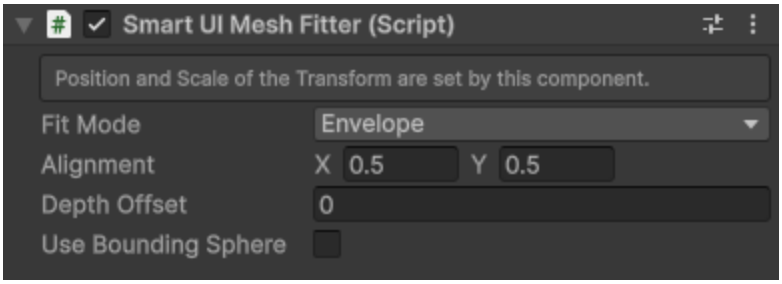
Demo scenes

- **Demo 13 - Mesh Fitter**: a lit tiled wall and an emblem that casts a shadow on it.
- **Demo 14 - World Follower**: objects in shop cards, each through a camera of its own.
- **Demo 15 - Character Framing**: a dress-up screen with a character in a room, equipment and framing bounds.

Smart UI Mesh Fitter component

Sizes and places a 3D object, for example a mesh or a whole model, inside the parent Rect Transform, so it scales with the UI like any UI element. Typical uses are a lit, normal-mapped menu background that reacts to lights, a 3D title that lifts off that background and casts a real shadow on it, spinning coins and items in the slots of a shop, and an item inspector where the player turns an item.

Add the component to the object and place the object under a Rect Transform, usually inside a UI Scaler. The canvas must be Screen Space Camera: Screen Space Overlay canvases do not draw 3D objects, and World Space canvases are not supported by the Smart UI Canvas Scaler.



Field	Description
Fit Mode	Fit: the object keeps its proportions and fits inside the parent. Envelope: the object keeps its proportions and covers the parent. Stretch: the object is stretched to the width and the height of the parent, for example a quad used as a background.
Alignment	Where the object sits inside the parent, from (0, 0) bottom left to (1, 1) top right. (0.5, 0.5) centers it, (0.5, 0) keeps it on the bottom edge. Not used by Stretch.
Depth Offset	How far the object is lifted from the parent toward the camera, in UI units. 0 puts the back of the object on the parent. It scales together with the UI, so a lifted object keeps the same look and the same shadow on every screen.
Use Bounding Sphere	Fit the sphere around the object instead of its box. The size then stays the same while a child of the object rotates, for example a spinning item in a shop slot. Suits roughly round objects: a wide object, such as a title, becomes much smaller. Not used by Stretch. Not needed with Smart UI Framing Bounds: their typed-in volume does not turn with the object, so turn it off.

How it works

After the UI Scalers are updated each frame, the component measures the bounds of all enabled Renderers on its GameObject and its children, scales the object so the bounds fit the parent, and moves

it so the bounds sit at the Alignment point with their back on the parent. The pivot of the model does not matter: a model with its pivot at its base is placed the same way as one with its pivot in its middle.

The Position and the Scale of the Transform are set by the component, so they cannot be edited by hand. Use Depth Offset instead of the Z position: UI Scalers scale the width and the height of their content but not its depth, so a Z position set by hand would stay the same while the object grows or shrinks with the UI. The component scales the depth together with the width and the height, so the object keeps its shape.

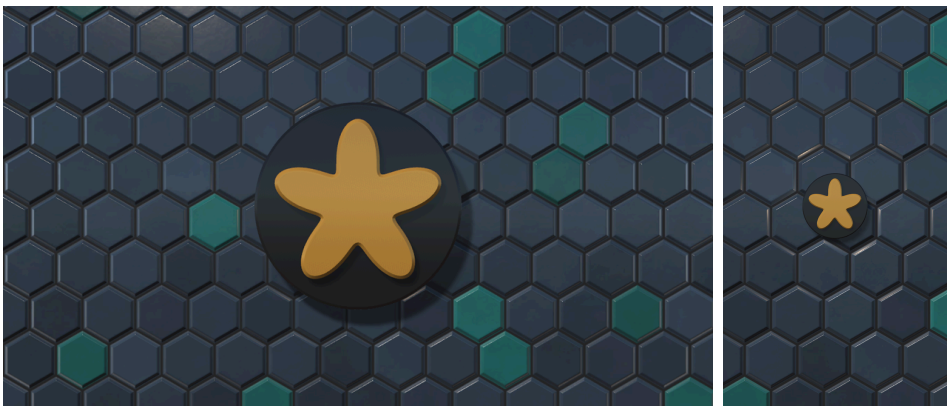
A 3D background that covers the screen

Use a quad with a lit material and the aspect ratio of its texture, and set its Mesh Fitter to Envelope, so the background covers the screen without being stretched. Unity's built-in Quad is square: use a mesh of your own, or put the Quad on a child object scaled to the texture's aspect ratio.

Place the quad directly under the Canvas, outside the UI Scaler. The Canvas covers the whole screen, so the background also covers the area outside the safe area. When the background has to stay inside a UI Scaler, for example in a prefab, place it under a Rect Transform with a Smart UI Expander that expands to all Canvas borders.

3D objects in front of the background, such as a title, can cast real shadows on it: give their Mesh Fitter a Depth Offset to lift them off the background, and let their Renderers cast shadows.

Demo 13 on a 16:9 and a 9:16 screen: the lit wall covers the screen at the proportions of its texture, and the emblem in front of it casts a shadow on the wall.



Good to know

- Rotate a child of the object, not the object itself. UI Scalers do not scale depth, so rotating the object around X or Y can skew it. Rotation around Z is fine.
- The Inspector warns when the object is not under a Rect Transform, when the canvas is Screen Space Overlay, when the object itself is rotated around X or Y and when no Renderer is found.

- The object is a normal 3D object in the scene: lights, shadows, layers and post processing work as usual. Make sure the camera's shadow distance covers the canvas plane distance.

Rotating and animated objects

- **Spinning items**, such as a coin next to a counter or an item in a shop slot: turn on Use Bounding Sphere and rotate a child object. The sphere looks the same from every side, so the size stays the same while the item turns.
- **Tilted titles and logos**: rotate a child object to the angle you want. The tilted object is measured as it is turned and fitted into its slot, and the camera of the canvas shows it in perspective.
- **Moving or animated models**, such as a swaying title or a waving mascot: animations change the measured Renderer bounds every frame, so the model would grow and shrink. Add Smart UI Framing Bounds and type in the volume the movement needs. The Mesh Fitter then fits that volume instead of measuring the Renderers, and the model keeps the same size.
- **An item inspector**: the player turns a child object by dragging, and the item is zoomed by resizing its slot.

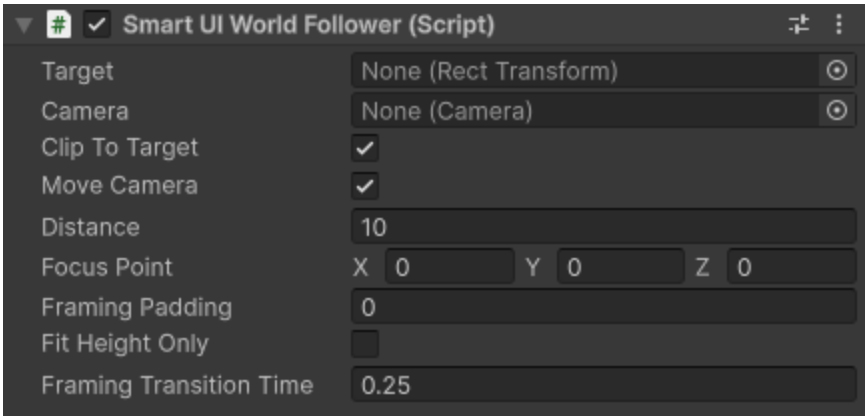
> **TIP**
>
> Not sure whether you need the Mesh Fitter or the Smart UI World Follower? See 3D objects in the UI.

Smart UI World Follower component

Shows a world object, for example a 3D model, inside a Rect Transform of your UI through a camera of its own. Typical uses are a character in a menu slot or an item in a shop slot that has to stay in place on every screen size.

Add the component to the world object and assign the Target and the Camera.

> **TIP**
>
> Not sure whether you need the World Follower or the Smart UI Mesh Fitter? See 3D objects in the UI.



Field	Description
Target	The Rect Transform the object is shown in, usually inside a UI Scaler. A Rect Transform with zero width or height works as a point: the object is placed on it and keeps the size the camera alone gives it.
Camera	The camera that shows the object. The follower drives its projection, and with Move Camera on also its position, so two followers must not share one camera.
Clip To Target	On: the camera only draws the on-screen part of the Target, so nothing outside the Target appears and a Target partly off-screen is cut, not squeezed. Turn it off when the camera must keep drawing its whole viewport: the main scene camera, or an Overlay camera in a URP camera stack, which ignores its own Viewport Rect. The camera then draws everything in its Culling Mask, so a camera that belongs to one object must show only that object.
Move Camera	On: the follower places the camera Distance behind the focus point, so the object is seen head-on. Off: the camera position and rotation are left to something else,

Field	Description
	for example Cinemachine or an animation, and the follower only shifts the projection so the focus point stays in the center of the Target.
Distance	With Move Camera on, how far the camera stays behind the focus point, along the camera's forward direction. Together with the camera's Field of View (or Size for an orthographic camera) it decides how big the object looks: the field of view covers the height of the Target.
Focus Point	The point of the object, in its local space, that is kept in the center of the Target. Use it when the object's pivot is not where its middle is, for example a character with its pivot at the feet.
Framing Padding, Fit Height Only, Framing Transition Time	Used when the object has Smart UI Framing Bounds. The projection is then zoomed so the framing bounds fit the Target, and their center replaces the Focus Point.

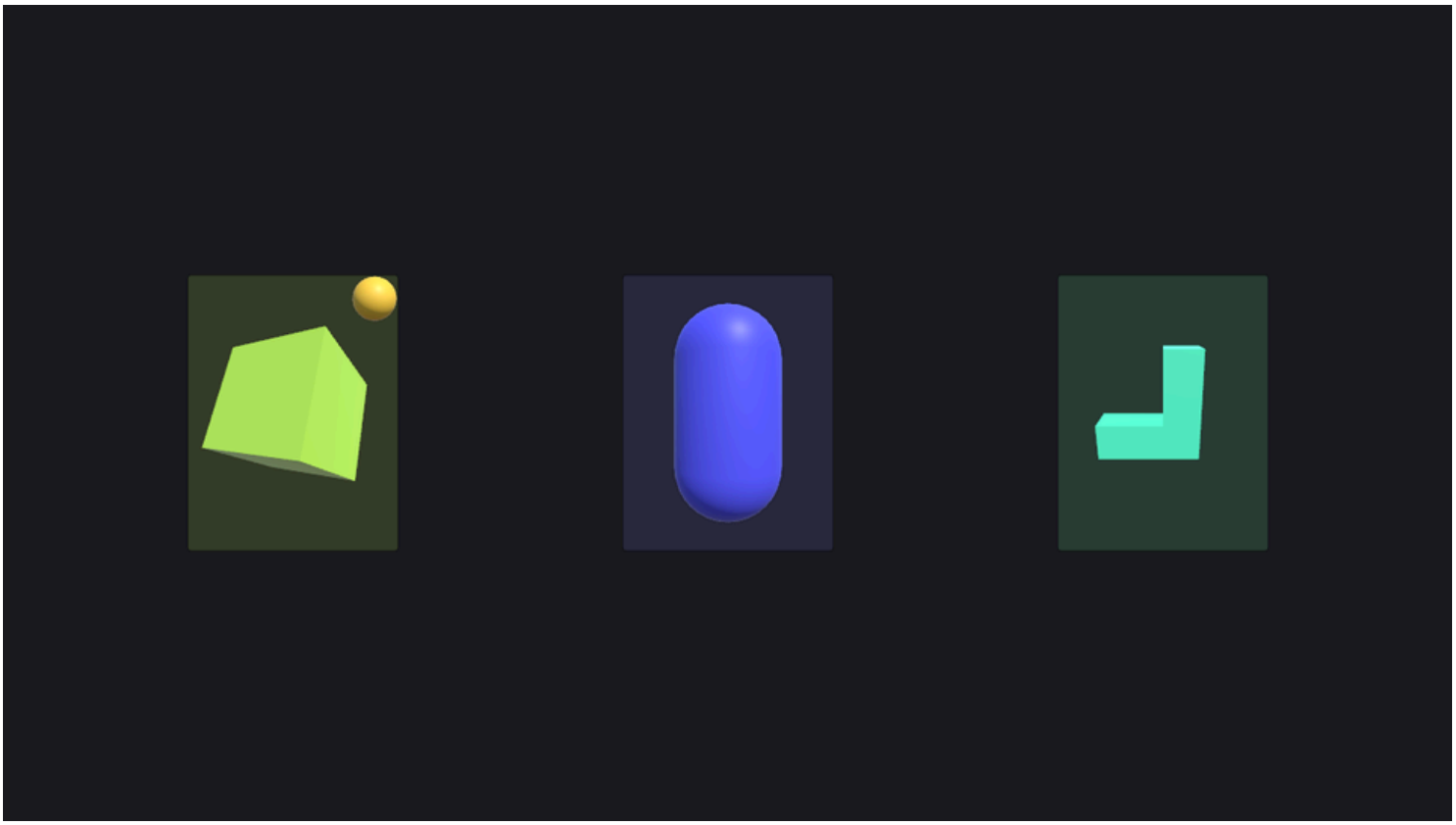
How it works

After the UI Scalers are updated each frame, the follower shifts the camera's projection so that the focus point lands in the center of the Target. With Move Camera on it also places the camera Distance behind the focus point first. With Clip To Target on, the camera's Viewport Rect is set to the on-screen part of the Target, so the object can be partly off-screen together with its Target without being squeezed.

Set up the camera like this:

- Clear Flags: Depth only, so the UI and the main camera image stay visible around the object.
- Culling Mask: only the layer of the object.
- A higher Depth than the main camera, so it draws after it. In a URP camera stack, make it an Overlay camera, turn Clip To Target off and keep its Viewport Rect the same as the main camera's.

Three shop cards, each showing a 3D model through a World Follower and a camera of its own. The models stay in their cards on every screen shape.



Render textures

When the Target is inside a Screen Space Camera canvas that renders into a Render Texture, for example with a custom UI Scaler, give the follower camera the same Target Texture. The follower then works in the texture's pixels, and the object becomes part of the image wherever that image is shown.

The follower camera clears only the depth, and only inside its own viewport. Make sure a camera with a lower Depth clears the whole texture every frame, for example the camera of that canvas with Clear Flags set to Solid Color. Otherwise pixels from earlier frames stay where the object was before its Target moved.

A character in the free space of a menu

In a lobby or character screen the 3D scene is usually the background, with the character standing in the space the menu panels leave free. That free space moves with the screen shape: on a phone or a tablet the panels take more or less room, and on an ultrawide screen the character would drift away from the panels. Put the follower on the main scene camera to keep the character in that space:

- Add a Rect Transform to the UI layout where the character should stand, for example between the side panels.
- Add the follower to the character, assign that Rect Transform as the Target and the main scene camera as the Camera.
- Turn Clip To Target off, so the camera still draws the whole screen.
- Turn Move Camera off and place the camera yourself, or with Cinemachine.

The follower then shifts the camera's picture so the character lands on the Target, like the lens shift of a real camera, without moving or turning the camera. With a Target of zero size only the position follows the layout and the camera keeps its own framing, which usually looks most natural. With a sized Target the character's height also follows the Target height, and the background shows a wider view when the Target is small.

Good to know

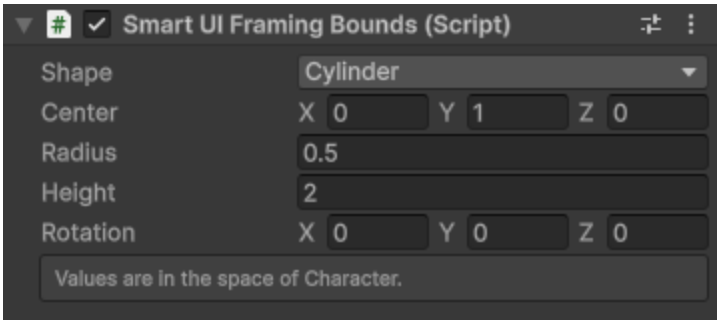
- Each follower uses a camera of its own, which is one more camera to render. A few followers are fine, for a grid of many items rendering each item once into a texture is cheaper.
- The follower camera sees the scene lights and post processing like any other camera. To light an object differently, use layers and the lights' culling masks.

> **NOTE**
>
> A Screen Space Overlay canvas is always drawn on top of all cameras. To place the object between UI layers, use Screen Space Camera canvases in a camera stack.

Smart UI Framing Bounds component

Describes the volume of a character or an object that a Smart UI World Follower keeps inside its Target, or that a Smart UI Mesh Fitter fits into its parent Rect Transform. Typical use is a character screen, a dress-up screen or a shop where the character must always be seen whole, between the menu panels, on every screen shape.

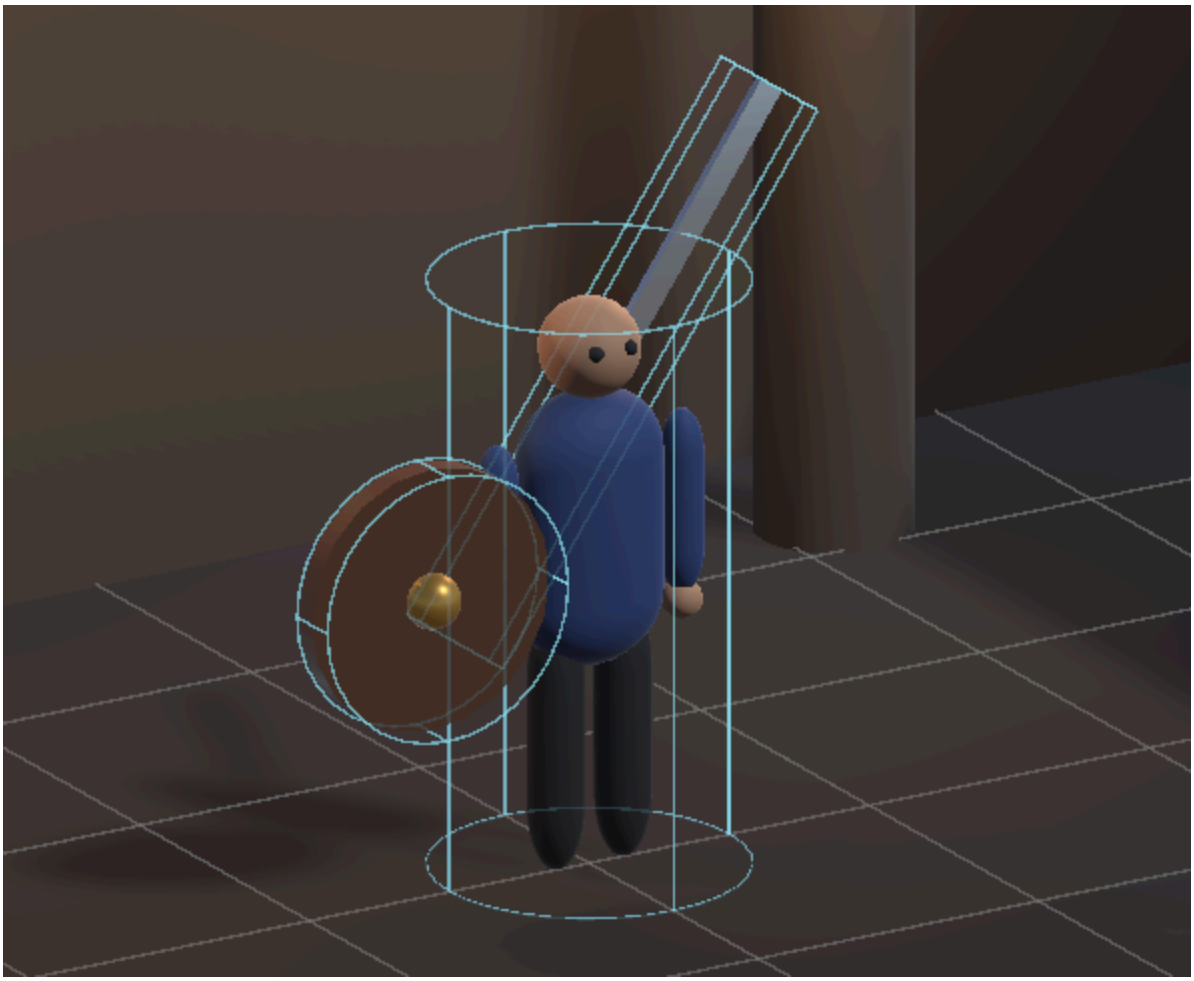
Add the component to the GameObject that has the Smart UI World Follower or the Smart UI Mesh Fitter and type in the volume the character needs, including the space its animations and poses use. The volume is drawn in the Scene view and can be resized with its handles.



Field	Description
Shape	Cylinder: a cylinder around the up axis. Its outline does not change while the character turns, so a character on a turntable keeps its size. Rotated, it fits round flat objects such as a shield. Box: a box, for objects that are not upright or not round. Sphere: its outline does not change however the object turns, for objects that tumble, such as a spinning item in a shop.
Center	Center of the volume.
Size	Size of the box. Used by Box.
Radius	Radius of the cylinder or the sphere. Used by Cylinder and Sphere.
Height	Height of the cylinder along its axis. Used by Cylinder.
Rotation	Rotation of the box or the cylinder in degrees, so the volume can lie along a diagonal object, such as a sword held at an angle, or face the camera like a shield, instead of wrapping it in a large upright volume. Used by Box and Cylinder.

All values are in the space of the nearest Smart UI World Follower or Smart UI Mesh Fitter, also when the component is on a child object. Framing bounds on a sword held in an animated hand therefore stay still while the hand moves, and the framing does not breathe with the animation.

The framing bounds of a character in the Scene view: an upright cylinder around the body, a box rotated along the greatsword and a cylinder rotated to face the camera around the shield.



With the Mesh Fitter

On a Smart UI Mesh Fitter, framing bounds replace the measured Renderer bounds: the Fit Mode, Alignment, Depth Offset and Use Bounding Sphere work on the typed-in volume. Use it for animated models in the UI, so they keep the same size while they move. The follower settings below are not used there.

Why the volume is typed in

The framing is not measured from the meshes. Animations, cloth and particles change the mesh bounds every frame, and a framing that follows them pumps in and out. A typed-in volume describes what you want to keep visible, and it only changes when you change it.

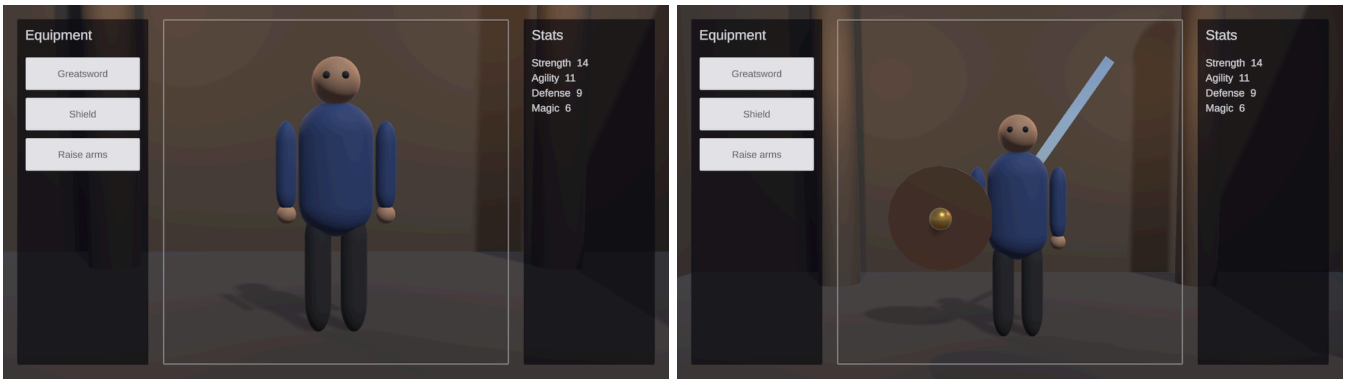
Equipment and poses

The follower combines all enabled framing bounds on its GameObject and its children:

- **Equipment**: give an item prefab framing bounds of its own, for example a box around a greatsword carried on the back. When the item is placed under the character, the framing widens to include it. When it is removed or disabled, the framing narrows again. No code is needed.
- **Poses**: the fields can be keyframed in an animation clip, for example a taller volume for a pose with raised arms.
- **Code**: set `Center`, `Size`, `Rotation`, `Radius`, `Height` or `Shape` at runtime, for example to frame only the head while a helmet is chosen.

The follower eases to every new framing over its Framing Transition Time, so equipping an item or changing a pose never makes the view jump.

The same character without and with the shield and the greatsword. The framing widens to include them, and the character stays inside its frame between the menu panels.



Follower settings

These fields of the Smart UI World Follower are used when framing bounds are found:

Field	Description
Framing Padding	Space between the volume and the edges of the Target, in UI units.
Fit Height Only	Only the height of the volume has to fit. Use it when the character should keep the same height and wide items may reach past the sides of the Target.
Framing Transition Time	Seconds the framing takes to follow a change. 0 follows at once. Changes in edit mode always apply at once.

With framing bounds the Focus Point is not used: the center of the combined volume is kept in the center of the Target.

Cinemachine and other camera rigs

The follower only zooms and shifts the camera's projection. The camera's position, rotation and lens stay with whatever drives the camera, such as Cinemachine, an animation or your own script. Turn Move Camera off on the follower and let the rig orbit, dolly or tilt the camera: the framing keeps the volume inside the Target from every angle.

Because the follower decides the size, moving the camera closer or changing its Field of View no longer makes the character bigger. It only changes the viewing angle and the strength of the perspective. To zoom in on a detail, for example the head while choosing a helmet, change the framing bounds to that detail instead.

> **NOTE**
>
> The volume is kept inside the Target, not in front of the scene. A camera that moves behind a wall still sees the wall. Use Cinemachine's deoccluder or keep the camera path clear.

Custom UI Scaler

A custom UI Scaler is needed when the parent Canvas has "Screen Space Camera" Render Mode and the attached camera renders into a Render Texture. The Smart UI Canvas Scaler component cannot know where the rendered image ends up on the screen, so a custom UI Scaler provides that information.

Demo 17 contains a complete implementation, `RenderTextureUIScaler`, for a canvas rendered into a Render Texture and shown by a RawImage. Use it as the reference for the steps below.

Implementation

Extend CoreUIScaler and implement these two methods:

Provide the screen space Rect of the area where the Render Texture is shown.

```
protected abstract Rect GetScreenRect()
```

Convert a screen space Rect to a world space Rect inside the Render Texture's Canvas.

```
protected abstract Rect ScreenRectToWorldRect(Rect screenRect)
```

Mark the class with `[ExecuteAlways]` so that the custom UI Scaler also works in Edit Mode, like the built-in components do.

Detecting changes

A UI Scaler resizes only when its Rect Transform has changed, which is what happens when the Canvas resizes. When the provided screen Rect can change on its own, for example because the RawImage moves on the screen, the custom UI Scaler has to request a resize itself. Override `OnPreWillRenderCanvases`, compare the current screen Rect with the previous one and set `transform.hasChanged = true` before calling the base method:

```
protected override void OnPreWillRenderCanvases()
{
        var screenRect = GetScreenRect();

        if (screenRect != _lastScreenRect)
        {
                _lastScreenRect = screenRect;
                transform.hasChanged = true;
        }
```

```
        base.OnPreWillRenderCanvases();
}
```

Inspector

The Group and Options fields of a UI Scaler are drawn by a custom inspector. To show them on a custom UI Scaler, add a nested editor class that extends `CoreUIScalerEditor` and append the custom fields after the base inspector:

```
#if UNITY_EDITOR
[UnityEditor.CustomEditor(typeof(MyUIScaler))]
class Editor : CoreUIScalerEditor
{
        public override void OnInspectorGUI()
        {
                base.OnInspectorGUI();

                serializedObject.Update();

UnityEditor.EditorGUILayout.PropertyField(serializedObject.FindProperty("_outputImage"));
                serializedObject.ApplyModifiedProperties();
        }
}
#endif
```

Safe area

The "Use Safe Area" option intersects the provided screen Rect with the screen's safe area, so it only makes sense when the screen Rect is in real screen coordinates. Demo 17 uses it this way: the inner UI Scaler only occupies the part of the image that is inside both the screen and the safe area. Turn the option off when the output is not shown directly on the screen.

Staying in sync with UI Scalers

UI Scalers are resized by SmartUIManager just before the canvases are rendered, after all `Update()` and `LateUpdate()` calls of the frame. Code that reads a UI Scaler's size or position in `Update()` therefore sees the values of the previous frame, which shows as a one frame lag whenever the screen, the safe area or the camera viewport changes.

To read UI Scaler sizes in the same frame they were updated, subscribe to AfterScalersUpdatedEvent. It is invoked every frame right after all UI Scalers have been updated, in Play Mode and in Edit Mode. The Smart UI Mesh Fitters and Smart UI World Followers have already placed their objects and cameras by then, so the event also sees their final positions.

```
using SmartUIScaler;
using UnityEngine;

public class ScalerFollower : MonoBehaviour
{
        [SerializeField]
        private RectTransform _scaler;

        private void OnEnable()
        {
                SmartUIManager.AfterScalersUpdatedEvent += OnAfterScalersUpdated;
        }

        private void OnDisable()
        {
                SmartUIManager.AfterScalersUpdatedEvent -= OnAfterScalersUpdated;
        }

        private void OnAfterScalersUpdated()
        {
                var size = _scaler.rect.size;
        }
}
```

Demo 16 shows the difference between a follower positioned in this event and one positioned in `Update()`. For the full order of a frame and what each point can rely on, see Lifecycle and update order.

Listening to one UI Scaler

To react to a single UI Scaler, add a listener with AddResizedListener(Action, bool). It is invoked only when that UI Scaler was resized, right after its Rect Transform was updated and before its child behaviours and the canvas layout are updated, so objects the listener activates are fitted in the same

frame. A Smart UI Aspect Switch on the UI Scaler has already switched its layouts by then. By default a listener added after the first resize is invoked immediately. Remove it with RemoveResized Listener(Action).

ScreenArea is the screen area, in screen pixels, that the UI Scaler was fitted to by its last resize, after the safe area was applied. Use it, for example, to choose a layout by the aspect ratio of the area instead of the whole screen.

> **NOTE**
>
> `SmartUIManager` is created automatically when the game starts and must not be added to a GameObject or destroyed manually.

Lifecycle and update order

This page explains when the plugin's components do their work during a frame, what happens when they are enabled or disabled, and which values your own code can read at each point.

Order within a frame

UI Scalers are not updated in `Update()` or `LateUpdate()`. SmartUIManager runs one pass just before the canvases are rendered. Unity's built-in Canvas Scaler uses this same point in the frame.

1. `Update()` and `LateUpdate()` of all scripts.
2. The scaler pass:
    1. The screen safe area is read.
    2. Every enabled UI Scaler and UI Scaler behaviour that needs it is resized. When a UI Scaler is resized, its own Rect Transform is updated first. Next, a Smart UI Aspect Switch on it switches layouts and invokes its events, and then the resized listeners added from code run. The Smart UI Expanders, Smart UI Size Limiters and Smart UI Aspect Fitters under it are resized last.
    3. The Smart UI Mesh Fitter and the Smart UI World Follower place their objects and cameras, after all UI Scalers have their final size.
    4. AfterScalersUpdatedEvent is invoked.
3. Unity rebuilds the canvas layout: Layout Groups, Content Size Fitters and graphics.
4. The frame is rendered.

Every step after `LateUpdate()` happens in the same frame, so the result is on screen in the frame where the screen changed.

What your code can rely on

Where your code runs	What it sees
`Awake()`, `OnEnable()`, `Start()` of the first frame	Nothing is fitted yet. UI Scalers still have the size they were saved with.
`Update()`, `LateUpdate()`	The sizes of the previous frame. A change of the screen in this frame is not applied yet.
A resized listener of a UI Scaler	The final Rect Transform of that UI Scaler, and the layouts of its Aspect Switch. The behaviours under it, and other UI Scalers, may not be updated yet.
A listener of the Smart UI Aspect Switch	The layouts are already switched.

Where your code runs	What it sees
`SmartUIManager.AfterScalersUpdatedEvent`	All UI Scalers and their behaviours are final, and the Mesh Fitters and World Followers have placed their objects and cameras. Rect Transforms placed by Layout Groups or Content Size Fitters are rebuilt after this event, so they may still have the values of the previous frame.

To read fitted sizes before the first frame is rendered, for example in `Start()`, call `Canvas.ForceUpdateCanvases()` first. It runs the scaler pass and the canvas layout rebuild immediately. Call it once and only where needed, as it updates every canvas.

See Staying in sync with UI Scalers for `AfterScalersUpdatedEvent`, the resized listener and `ScreenArea`.

Component lifetime

- **Awake:** a UI Scaler finds its parent and child UI Scaler behaviours. Nothing is resized.
- **OnEnable:** a UI Scaler starts taking part in the scaler pass and takes control of its Rect Transform.
- **First scaler pass:** a newly enabled UI Scaler is fitted in the first scaler pass after it was enabled, in the same frame. It is never fitted in `Awake()` or `Start()`.
- **OnDisable:** the UI Scaler leaves the scaler pass and releases its Rect Transform, which keeps its last fitted values. Nothing else moves.
- **Enabled again:** the UI Scaler is fitted in the next scaler pass if its options, its group's options or the screen changed while it was disabled.

The Smart UI Expander, Smart UI Size Limiter and Smart UI Aspect Fitter follow the same rules. The Smart UI Mesh Fitter and the Smart UI World Follower start working in the first scaler pass after they are enabled, before `AfterScalersUpdatedEvent` is invoked. When disabled, the World Follower restores the projection and viewport of its camera. Smart UI Framing Bounds do not update anything themselves. They are read by the World Follower or Mesh Fitter they belong to, in that component's next update.

What triggers a resize

A UI Scaler is resized in the scaler pass only when something changed, otherwise the pass only checks that nothing changed. These changes cause a resize:

- the screen resolution or the camera viewport of the Canvas;
- the screen safe area, also when only the safe area changes, for example when a phone is turned upside down;
- its options in the Inspector, or the options of its UI Scaler group, also when they are changed at runtime;

- its Transform or its parent, for example when it is moved to another parent;
- in the editor, the settings of the Smart UI Scaler window.

To make a UI Scaler refit in the next scaler pass for any other reason, set `transform.hasChanged = true` on it.

Layouts of the Smart UI Aspect Switch in the first frame

The Smart UI Aspect Switch switches layouts inside the first scaler pass, before the first frame is rendered. The correct layout is visible from the first frame.

A layout that was saved active is still active until that pass. Its scripts receive `Awake()`, `OnEnable()`, `Start()` and one `Update()` and `LateUpdate()` before it is deactivated, even when another variant wins on this screen. The layout that is activated by the switch receives `Awake()` and `OnEnable()` during the scaler pass and `Start()` and its first `Update()` in the next frame.

When the scripts in your layouts must only run in the layout that is used, deactivate all layouts before saving the scene. The Aspect Switch then only activates the layout that is used. In the "Fitted Reference Resolution" and "Reference Resolution" scaling modes, the UI Scaler then looks empty until a layout is shown with an "Edit Layout" button.

Edit Mode

All components run in Edit Mode too, so the layout looks the same as in Play Mode. The scaler pass runs whenever the editor redraws the canvases, for example when the Game view is resized or a value is changed in the Inspector. The "Scaling Mode" of the Smart UI Scaler window decides the size of UI Scalers in Edit Mode, and the Smart UI Aspect Switch only switches layouts in the "Preview" scaling mode.

In Play Mode the World Follower smooths its framing changes with unscaled time, so they also play while `Time.timeScale` is 0, for example in a pause menu.

Scripting define

Smart UI Scaler adds the `SMART_UI_SCALER` scripting define symbol to the Player Settings of every platform when the editor loads it. Code that should work with and without the plugin can check it, for example a shared HUD script or a plugin of your own that supports Smart UI Scaler as an option:

```
#if SMART_UI_SCALER
using SmartUIScaler;
#endif
using UnityEngine;

public class HudLayout : MonoBehaviour
{
        private void OnEnable()
        {
#if SMART_UI_SCALER
                SmartUIManager.AfterScalersUpdatedEvent += OnScalersUpdated;
#endif
        }

        private void OnDisable()
        {
#if SMART_UI_SCALER
                SmartUIManager.AfterScalersUpdatedEvent -= OnScalersUpdated;
#endif
        }

        private void OnScalersUpdated()
        {
        }
}
```

The symbol is added once to each platform in Edit > Project Settings > Player > Other Settings > Scripting Define Symbols, next to the symbols that are already there. When it is already set, nothing is changed. Adding it makes Unity compile the scripts once more after the plugin is imported.

If your code is in an assembly with its own assembly definition, add a reference to the `SmartUIScaler` assembly definition, so the code inside `#if SMART_UI_SCALER` can use the plugin's types.

NOTE

Unity does not remove a scripting define symbol when the plugin that added it is deleted. After removing Smart UI Scaler from a project, also remove `SMART_UI_SCALER` from the Scripting Define Symbols of each platform, otherwise the code inside `#if SMART_UI_SCALER` is still compiled and cannot find the plugin.

Demo scenes

The plugin ships with demo scenes under `Assets/Plugins/SmartUIScaler/Demos`, ordered from the most basic concept to the advanced ones. Every scene has a description panel that explains what to look at. Hide or show it with Alt+Shift+I in the Editor, in Edit Mode and in Play Mode. The shortcut can be changed in "Edit" > "Shortcuts" under "Smart UI Scaler". In Play Mode the button on its top right corner does the same. The choice is remembered for all demo scenes.

Most demos are meant to be explored without entering Play Mode: open "Tools" > "Smart UI Scaler", set the Scaling Mode to "Preview" and change the Game view aspect ratio.

Demos 13 to 15 use materials of the Built-in Render Pipeline. In a URP or HDRP project convert them with the render pipeline's material converter, otherwise they are drawn in magenta.

Demo	Scene	What it shows
1	Single Scaler	One Smart UI Canvas Scaler with Expand Progress 1 that always fills the safe area. Markers stay anchored to its corners and edges while the aspect ratio changes.
2	Reference Resolution and Safe Area	The same HUD layout in a 1600 x 900 and in a 900 x 1600 Reference Resolution, both with Expand Progress 0, next to Use Safe Area on versus off. Enable "Simulate Safe Area" and "Show Safe Area Border" in the editor window to see the difference.
3	Alignment	Nine Smart UI Canvas Scalers, one per Alignment value, each in its own Screen Space Camera canvas. In Play Mode every viewport animates between wide and tall.
4	Expand Progress	Three centered Smart UI Canvas Scalers with Expand Progress 0, 0.5 and 1.
5	Content Scale	Two Smart UI Canvas Scalers with Content Scale 1 and 0.5. The boxes shrink, the covered screen area stays the same.
6	UI Scaler Groups	A HUD and a portrait, two Smart UI Canvas Scalers bound to the same group. In Play Mode press Escape to open a menu whose sliders change the group's Expand Progress and Content Scale. The settings persist through PlayerPrefs.
7	Smart UI Expander	A header background that uses Smart UI Expander to reach the canvas borders beyond the safe area.

Demo	Scene	What it shows
8	Smart UI Size Limiter	A lobby layout kept between a min aspect ratio of 1.8 and a max aspect ratio of 2.5.
9	Nested Behaviours	A Smart UI Expander and a Smart UI Size Limiter nested inside a Smart UI Canvas Scaler.
10	Aspect Fitter	The same picture inside three frames through Smart UI Aspect Fitters: Envelope Progress 0 shows the whole picture, Envelope Progress 1 covers the frame, and Envelope Progress 1 with a focal point keeps the character in view. See Smart UI Aspect Fitter component.
11	Aspect Switch	One Smart UI Canvas Scaler with a Smart UI Aspect Switch that activates a wide, a standard or a tall layout depending on the aspect ratio. See Smart UI Aspect Switch component.
12	Split Screen	Two split screen cameras, each with its own Smart UI Canvas Scaler fitted to its camera viewport, so both halves keep their layout on every screen shape. A toggle switches between scalers with Safe Area Symmetry None and Horizontal: with a simulated left safe area, None shrinks only the left scaler, Horizontal makes the right one give up the same width, so both keep the same scale and stay centered.
13	Mesh Fitter	A lit menu background built from 3D objects: a normal-mapped tiled wall that a Smart UI Mesh Fitter in Envelope mode scales to cover the screen, and an emblem lifted off it with Depth Offset so it casts a shadow. Press Play to move the light. See Smart UI Mesh Fitter component.
14	World Follower	Three shop cards and a gold sphere, each showing a world object through a Smart UI World Follower: a centered cube, a capsule with a raised Focus Point, an object whose camera is moved by a script with Move Camera off, and the sphere on a point target. See Smart UI World Follower component.
15	Character Framing	A dress-up screen: a character in a room stays inside the area between the menu panels through a Smart UI World Follower with Smart UI Framing Bounds. Equipping items and raising the arms widen the framing smoothly, and a swaying camera shows that the framing holds from every angle. Press Play. See Smart UI Framing Bounds component.

Demo	Scene	What it shows
16	After Scalers Updated Event	A Smart UI Canvas Scaler in an animated camera viewport, followed by two frames on another canvas: one positioned in `SmartUIManager.AfterScalersUpdatedEvent` stays in sync, one positioned in `Update()` trails a frame behind. See Staying in sync with UI Scalers.
17	Custom UI Scaler	A canvas rendered into a Render Texture and shown by a RawImage, with a custom UI Scaler inside that only uses the part of the image inside the screen and the safe area. See Custom UI Scaler.

NOTE

The demos use TextMeshPro for their description panels. If the TextMeshPro essential resources are not in your project yet, Unity offers to import them when a demo scene is opened.

Release notes

1.2.0

New

- Smart UI Aspect Switch component switches between layouts, such as wide, standard and tall, by the aspect ratio of the UI Scaler's screen area, with events per layout and a listener for code. Each layout can be edited in its own shape: the inspector sizes the UI Scaler as on a screen with the layout's aspect ratio. See Smart UI Aspect Switch component.
- Smart UI Aspect Fitter component keeps images and backgrounds at their aspect ratio, anywhere between fitting inside the parent and covering it, with alignment, a focal point and the aspect ratio taken from the Image or RawImage. See Smart UI Aspect Fitter component.
- Smart UI Size Limiter has an Alignment field that places the limited Rect Transform inside the parent, for example on the bottom right corner instead of the center. Existing Size Limiters stay centered.
- The Smart UI Scaler window has a "Show Safe Area Border" toggle that outlines the safe area in the Game view and shades the margins outside it, with a "Shade Unsafe Area" toggle for the shade. See Smart UI Scaler editor window.
- Smart UI World Follower component shows a world object, such as a 3D model, inside a Rect Transform of the UI through a camera of its own, also in camera stacks and partly off-screen. See Smart UI World Follower component.
- Smart UI Mesh Fitter component sizes and places a 3D object inside a Rect Transform, so lit meshes, such as a menu background and an emblem that casts a shadow on it, scale with the UI. See Smart UI Mesh Fitter component.
- Smart UI Framing Bounds component describes the volume of a 3D object, such as a character, an item or a title, as a cylinder, a box or a sphere that can be rotated. A Smart UI World Follower keeps that volume inside its Target, with Framing Padding, Fit Height Only and smooth transitions, and a Smart UI Mesh Fitter fits it instead of measuring the Renderers, so moving and animated models keep the same size. Equipment with framing bounds of its own widens the framing, and the camera can be driven by Cinemachine or any other rig. See Smart UI Framing Bounds component.
- "Safe Area Symmetry" option for UI Scalers and UI Scaler groups mirrors the safe area inset of one side to the opposite side, so a UI Scaler stays centered and split screen UI Scalers keep the same scale. See Smart UI Canvas Scaler component.
- `CoreUIScaler.ScreenArea` reports the screen area a UI Scaler was fitted to, and `AddResizedListener` / `RemoveResizedListener` notify code when a UI Scaler was resized, in the same frame. See Staying in sync with UI Scalers.
- The `SMART_UI_SCALER` scripting define symbol is added to every platform when the plugin is imported, so code can check for the plugin with `#if SMART_UI_SCALER`. See Scripting define.
- New demo scenes: Aspect Fitter, Aspect Switch, Mesh Fitter, World Follower and Character Framing. The Split Screen demo shows Safe Area Symmetry. The description panel of every demo can be hidden with Alt+Shift+I. See Demo scenes.

Changes

- The Smart UI Scaler component is renamed to Smart UI Canvas Scaler (class `SmartUICanvasScaler`). Existing scenes and prefabs keep working without changes.
- All components are public, so they can be added with `AddComponent<T>()` and found with `GetComponent<T>()`. Options without a public property are set in the Inspector.
- The Smart UI Scaler window keeps its settings when scripts are recompiled, is split into sections and has tooltips on every field. The Area fields are disabled outside "Preview" and Play Mode, and while "Simulate Safe Area" is off. An Area field cannot go past the opposite edge of the Game view. Presets no longer have a Scaling Mode: applying one switches to "Preview". The "PreviewPresets" asset is renamed to "No Safe Area Presets". In Preview mode the Scene view shows a reminder with an "Exit Preview" button.

Fixes

- A UI Scaler that was disabled while its group options changed now applies the new options when it is enabled again.
- Adding or removing a listener of `SmartUIManager.AfterScalersUpdatedEvent` from inside a listener no longer causes an error.
- An exception thrown by a group listener is logged and no longer stops the other listeners.
- A Smart UI Expander, Size Limiter or nested UI Scaler whose component is disabled is no longer resized by the UI Scaler above it. It keeps its Rect Transform until it is enabled again.

> **NOTE**
>
> When upgrading from 1.1.0, delete the `Assets/Plugins/SmartUIScaler` folder before importing 1.2.0. Importing a package never deletes files, and files that were moved or renamed in 1.2.0, such as the renumbered demos and the Smart UI Canvas Scaler script, would otherwise stay in the project next to their new copies. Your scenes, prefabs and UI Scaler group assets outside that folder keep working: they find the new scripts after the import.

1.1.0

New

- Unity 6 support, tested up to Unity 6000.6. The minimum version stays Unity 2022.3.0.
- `SmartUIManager.AfterScalersUpdatedEvent` is invoked every frame after all UI Scalers are updated. Use it to read UI Scaler sizes in the same frame. See Staying in sync with UI Scalers.
- UI Scaler Group scriptable objects have a "Save Id" field that identifies the group in saved data. It is generated automatically and kept unique by the editor.

- UI Scaler Groups Persistence Manager also saves when the application is paused, so settings survive on mobile when the app is closed from the background.
- Twelve demo scenes, from the basic concepts to split screen and custom scalers. See Demo scenes.

Changes

- UI Scalers refit when the safe area changes without a resolution change, for example when a phone with a camera cutout is turned upside down.
- Content Scale is limited to a minimum of 0.1 and Expand Progress to the range 0 to 1.
- The `Scripts/Internal` folder of the plugin is now `Scripts/Core`. If an empty `Scripts/Internal` folder is left in the project after upgrading, delete it.

> **WARNING**
>
> Settings saved by version 1.0 through the UI Scaler Groups Persistence Manager are not restored by 1.1.0. Each group starts from its "Default Options" once and is saved in the new format from then on.

1.0

Initial release.